rent_prices_eda

March 4, 2026

Rent price prediction in Berlin

Further Data Exploratory Analysis

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

[2]:
```
df = pd.read_csv('rent_listings.csv')
df.shape
```

[2]:
```
(15634, 23)
```

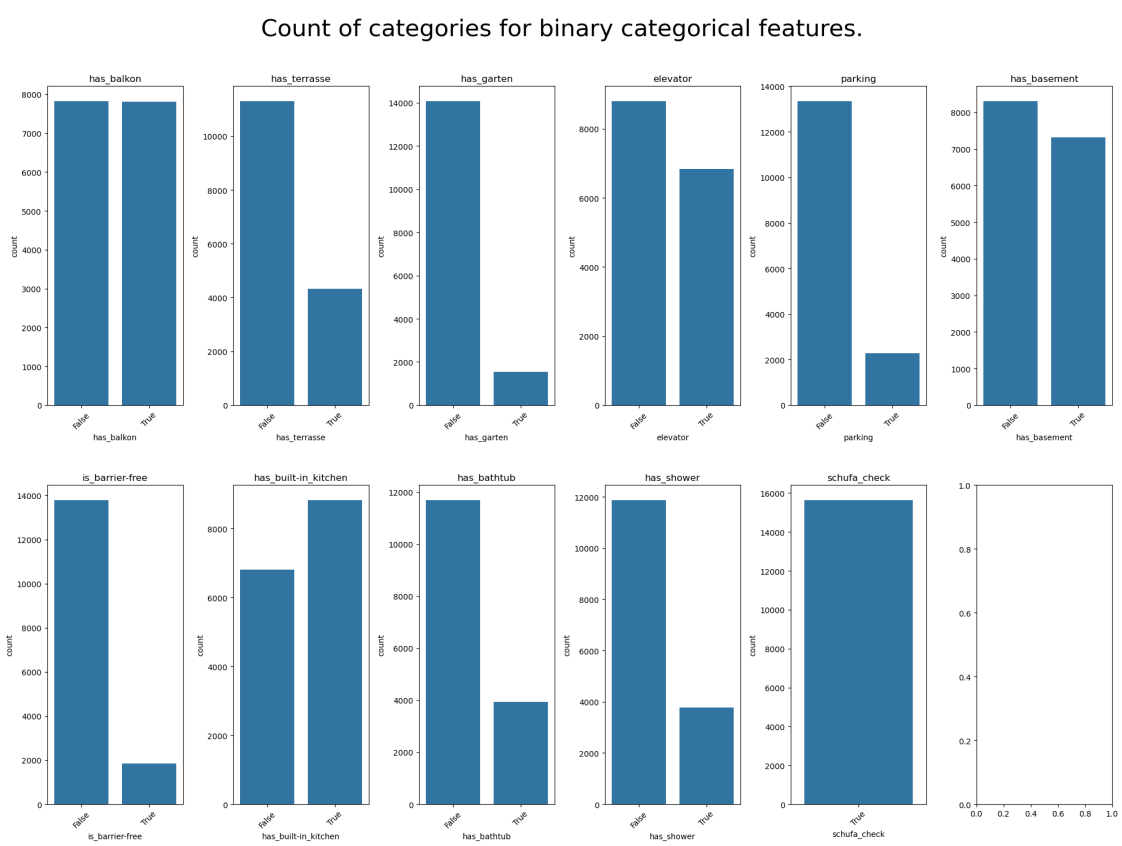
[3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15634 entries, 0 to 15633
Data columns (total 23 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   listing_url           15634 non-null  object
 1   cold_rent             15634 non-null  float64
 2   area                  15634 non-null  float64
 3   rooms                 15634 non-null  float64
 4   floor                 13686 non-null  float64
 5   free_from             15634 non-null  object
 6   has_balkon            15634 non-null  bool
 7   has_terrasse          15634 non-null  bool
 8   has_garten            15634 non-null  bool
 9   elevator              15634 non-null  bool
 10  parking               15634 non-null  bool
 11  has_basement          15634 non-null  bool
 12  is_barrier-free       15634 non-null  bool
 13  has_built-in_kitchen  15634 non-null  bool
 14  has_bathtub           15634 non-null  bool
 15  has_shower            15634 non-null  bool
 16  flooring_type         15634 non-null  object
 17  energy_source         15634 non-null  object
```

```
 18  heating_type          15634 non-null  object
 19  property_condition    15634 non-null  object
 20  year_built            7688 non-null   float64
 21  schufa_check          15634 non-null  bool
 22  district              15634 non-null  object
dtypes: bool(11), float64(5), object(7)
memory usage: 1.6+ MB
```

Part 1: Univariate Analysis

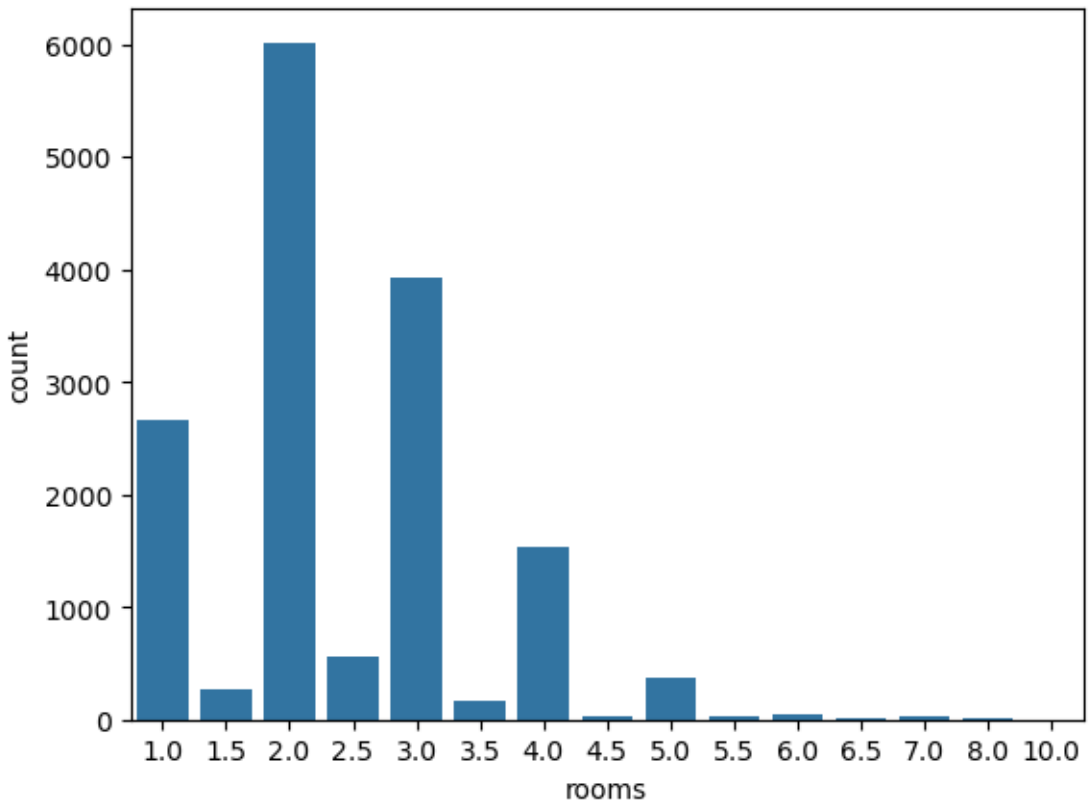
[4]:
```
features = ['has_balkon', 'has_terrasse',␣
 ↪'has_garten','elevator','parking','has_basement','is_barrier-free','has_built-in_kitchen','

fig, axes = plt.subplots(nrows=2, ncols=6, figsize=(20, 15))
fig.suptitle("Count of categories for binary categorical features.",size=30)

for ax, feature in zip(axes.flatten(), features):
    sns.countplot(data=df, x=feature, ax=ax)
    ax.set_title(feature)
    ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.subplots_adjust(hspace=0.25, top=0.9)
plt.show()
```



The majority of the apartments posses a built-in kitchen, which is a very important feature for an appartment. Half of them have a balcony and also features like an elevator and a basementare present in a considerable amount of apartments. While the rest of the features are not that common, but still present in some of the apartments. 100% of the apartments requre a schufa_check making this feature useless, therefore we have to drop it. We used to have 'false' values of this features, but appearenlty they were dropped during the data cleaning proccess.

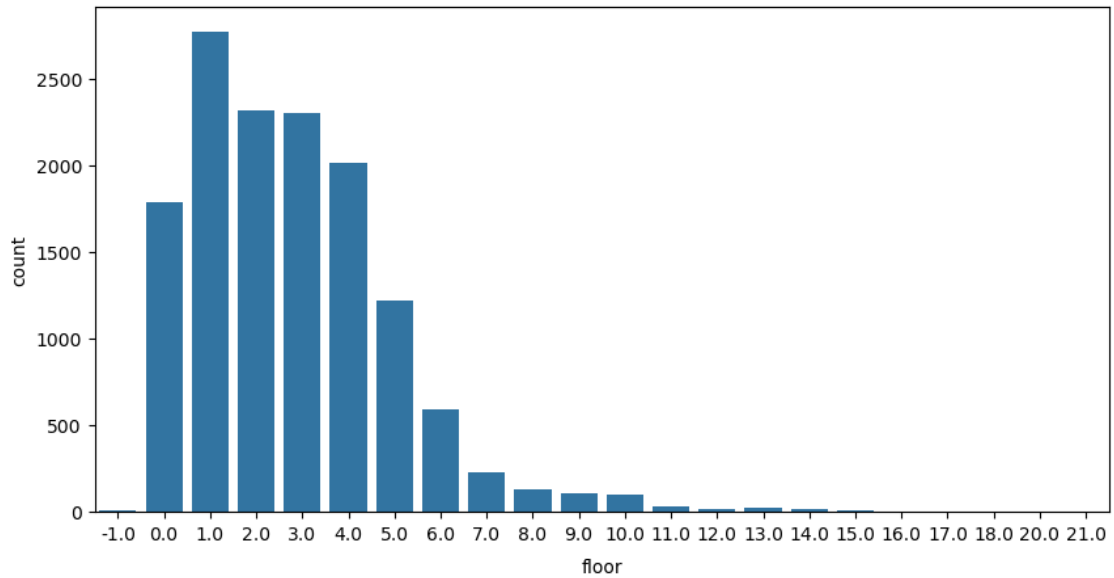
[5]:
```
df = df.drop(columns="schufa_check")
```

[6]:
```
sns.countplot(data=df, x="rooms")
plt.show()
```



Most of the apartments offer 2 rooms, followed by 3 rooms, 1 and 4 rooms.

[7]:
```
fig, ax = plt.subplots(figsize=(10, 5))
sns.countplot(data=df, x="floor", ax=ax)

ax.xaxis.labelpad = 8
plt.show()
```



[8]:
```
df["floor"].value_counts()
```

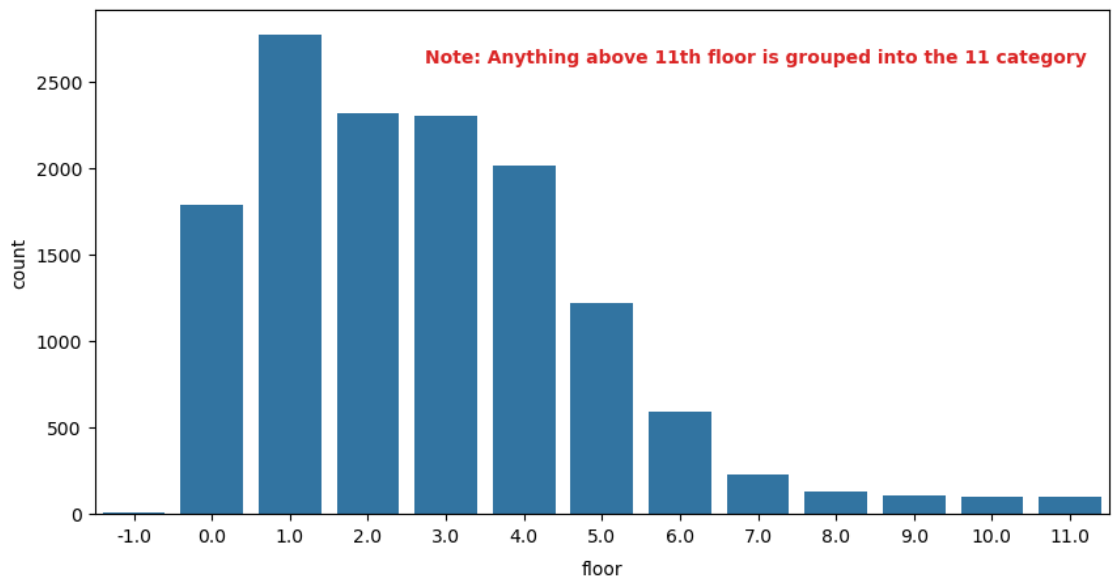
[8]:
```
floor
 1.0     2776
 2.0     2316
 3.0     2306
 4.0     2016
 0.0     1787
 5.0     1219
 6.0      594
 7.0      226
 8.0      128
 9.0      106
 10.0      99
 11.0      35
 13.0      23
 12.0      14
 14.0      13
-1.0       10
 15.0       6
 17.0       4
 16.0       3
 18.0       3
 21.0       1
 20.0       1
Name: count, dtype: int64
```

Encoding one additional dummy variable for categories that have less very little occurances such as

any floor above 10th floor would cause more noise rather than help the model learn new information. The best solution for this is to group these values together as one.

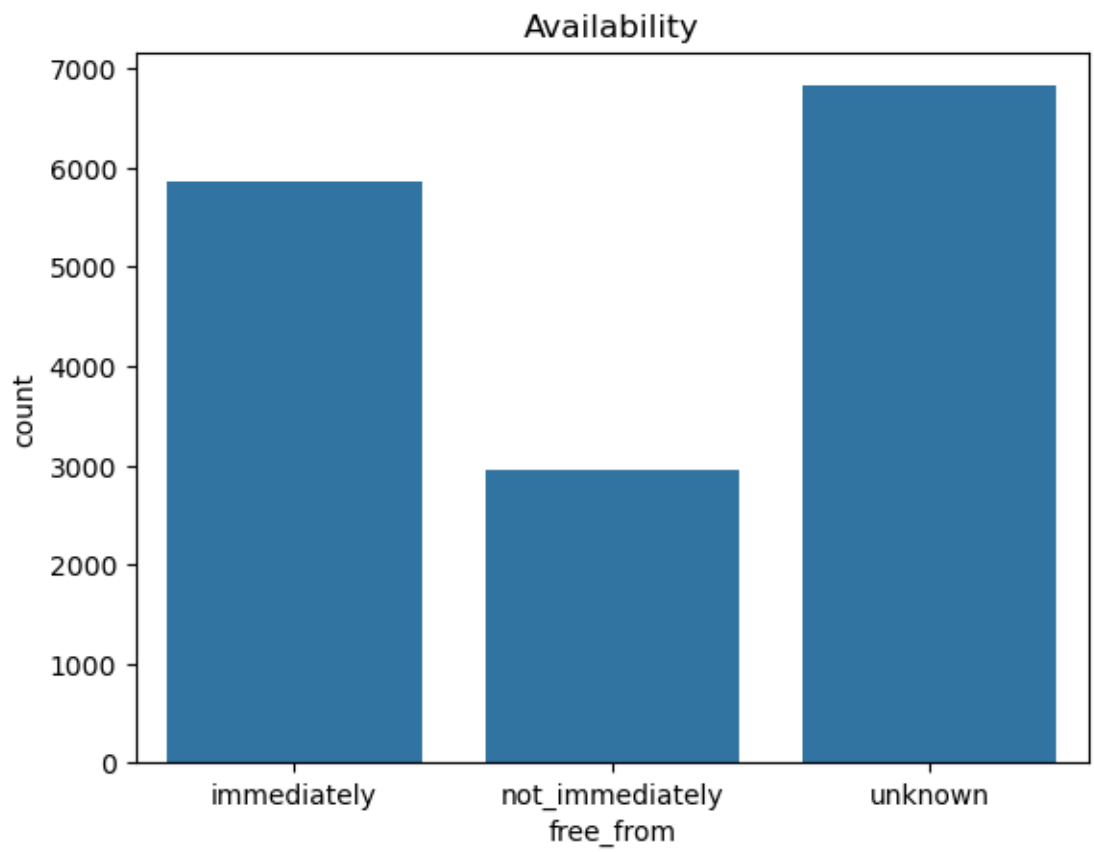
[9]:
```
# we group anything above the 10th floor as 11th floor
df['floor'] = df['floor'].apply(
    lambda x: x if pd.isna(x) or x <= 10 else 11
)
```

[10]:
```
fig, ax = plt.subplots(figsize=(10, 5))
sns.countplot(data=df, x="floor", ax=ax)
ax.xaxis.labelpad = 8
fig.text(
    0.63, 0.8,
    "Note: Anything above 11th floor is grouped into the 11 category",
    ha='center',
    color="#db2525",
    fontweight='bold'
)
plt.show()
```



Now we have a much more fair distribution of values, with the most values being between 1st and 4th floor. Also a considerable amount of apartments offer appartments in the ground and 5th floor.

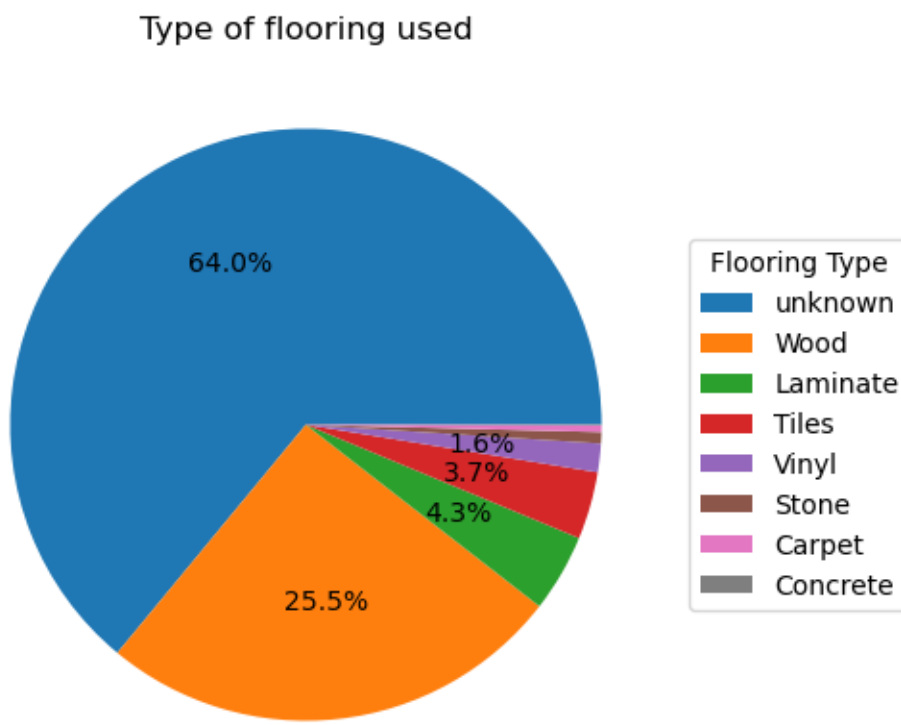
[11]:
```
sns.countplot(data=df, x="free_from")
plt.title("Availability")
plt.show()
```



If we exclude the samples where we don't have information, in about 2/3 of the remaining apartments it is possible to move in immediately.

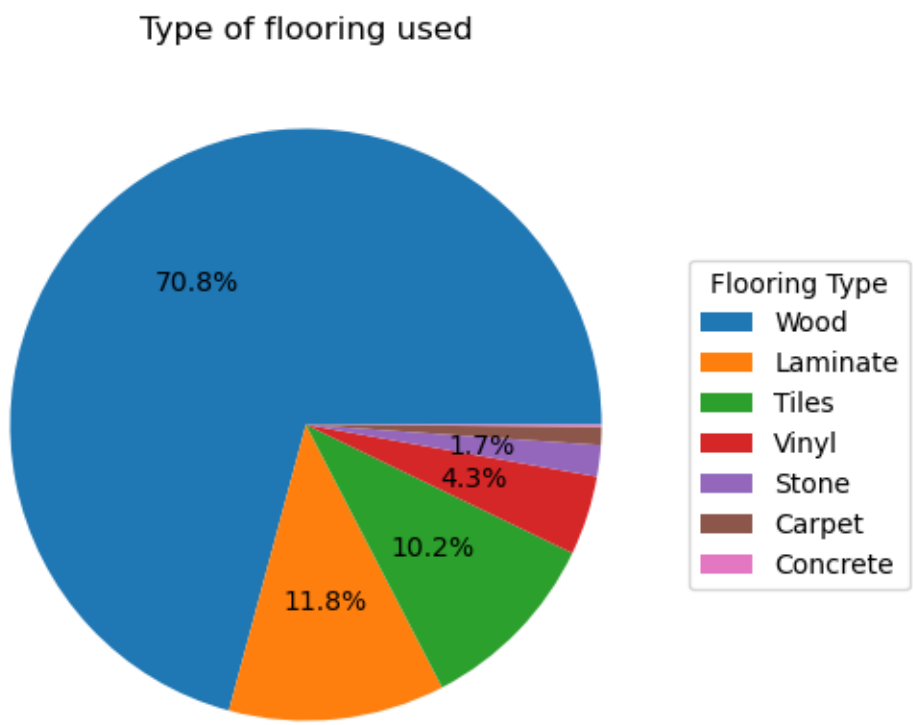
[12]:
```
counts = df["flooring_type"].value_counts()

plt.figure(figsize=(5,5))

plt.pie(
    counts.values,
    autopct=lambda p: f'{p:.1f}%' if p > 1.5 else '',
    labeldistance=1.2
)

plt.legend(
    counts.index,
    title="Flooring Type",
    loc="center left",
    bbox_to_anchor=(1, 0.5)
)
```

```
plt.title("Type of flooring used")
plt.show()
```



[13]:
```
# If we exclude unknown
counts = df[df["flooring_type"] != "unknown"]["flooring_type"].value_counts()

plt.figure(figsize=(5,5))

plt.pie(
    counts.values,
    autopct=lambda p: f'{p:.1f}%' if p > 1.5 else '',
    labeldistance=1.2
)

plt.legend(
    counts.index,
    title="Flooring Type",
    loc="center left",
    bbox_to_anchor=(1, 0.5)
)
```

```
plt.title("Type of flooring used")
plt.show()
```



From the graph we notice that wood is pretty popular compared to the rest. While other types can still be present but in a narrow range of apartments.

[14]:
```
# Let's take a further look into 'Concrete'
df['flooring_type'].value_counts()
```

[14]:
```
flooring_type
unknown     10006
Wood         3985
Laminate      665
Tiles         576
Vinyl         244
Stone          96
Carpet         56
Concrete        6
Name: count, dtype: int64
```

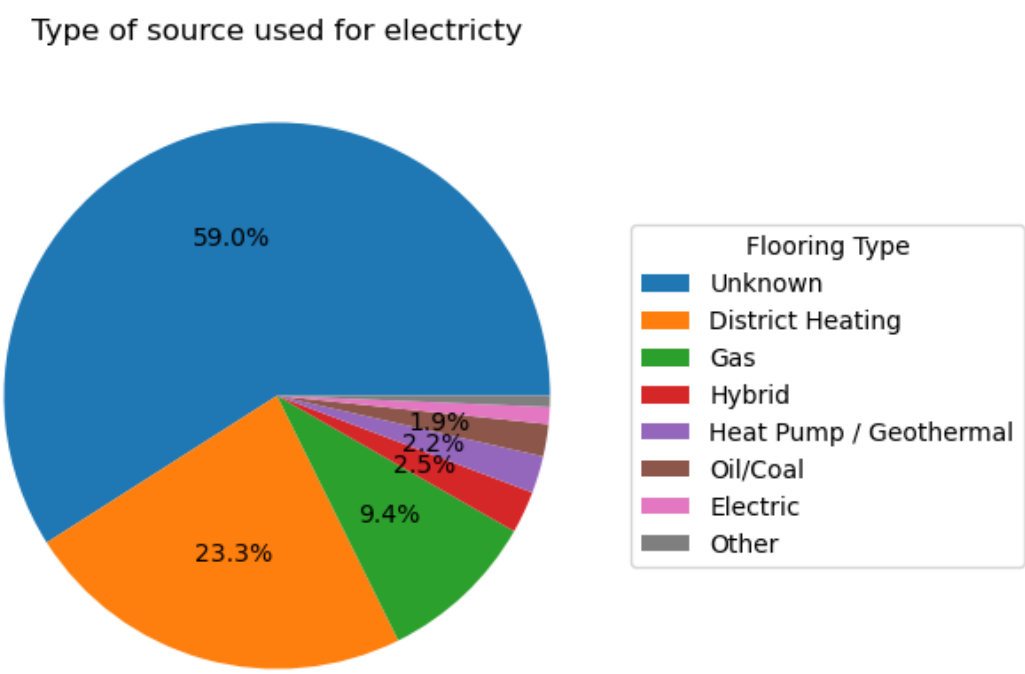
[15]:
```
# only 6 samples that have this category. Let's inspect those values first.
df[df["flooring_type"] == "Concrete"]
```

[15]:
```
                                          listing_url  cold_rent  area  \
1738  https://www.immowelt.de/expose/4261a90a-746d-4…     531.58  63.9
2058  https://www.immowelt.de/expose/b9b8d537-b7c0-4…     531.58  63.9
2069  https://www.immowelt.de/expose/684233d3-9f7a-4…     475.90  64.3
2555  https://www.immowelt.de/expose/684233d3-9f7a-4…     475.90  64.3
3356  https://www.immowelt.de/expose/617ad95e-a353-4…     341.86  39.6
5494  https://www.immowelt.de/expose/4d97a629-4fc1-4…     531.58  63.9

      rooms  floor        free_from  has_balkon  has_terrasse  has_garten  \
1738    2.0    5.0  not_immediately        True         False       False
2058    2.0    5.0  not_immediately        True         False       False
2069    2.0    5.0      immediately        True         False       False
2555    2.0    4.0      immediately        True         False       False
3356    1.0    3.0      immediately        True         False       False
5494    2.0    1.0  not_immediately        True         False       False

      elevator  …  is_barrier-free  has_built-in_kitchen  has_bathtub  \
1738      True  …            False                 False        False
2058      True  …            False                 False        False
2069      True  …            False                 False         True
2555      True  …            False                 False         True
3356      True  …            False                 False         True
5494      True  …            False                 False         True

      has_shower  flooring_type     energy_source heating_type  \
1738        True       Concrete  District Heating      Unknown
2058        True       Concrete  District Heating      Unknown
2069       False       Concrete  District Heating      Unknown
2555       False       Concrete  District Heating      Unknown
3356       False       Concrete  District Heating      Unknown
5494       False       Concrete  District Heating      Unknown

     property_condition year_built  district
1738   Needs Renovation     1971.0  Wittenau
2058   Needs Renovation     1971.0  Wittenau
2069          Renovated     1970.0  Wittenau
2555          Renovated     1970.0  Wittenau
3356          Renovated     1970.0  Wittenau
5494   Needs Renovation     1971.0  Wittenau

[6 rows x 22 columns]
```

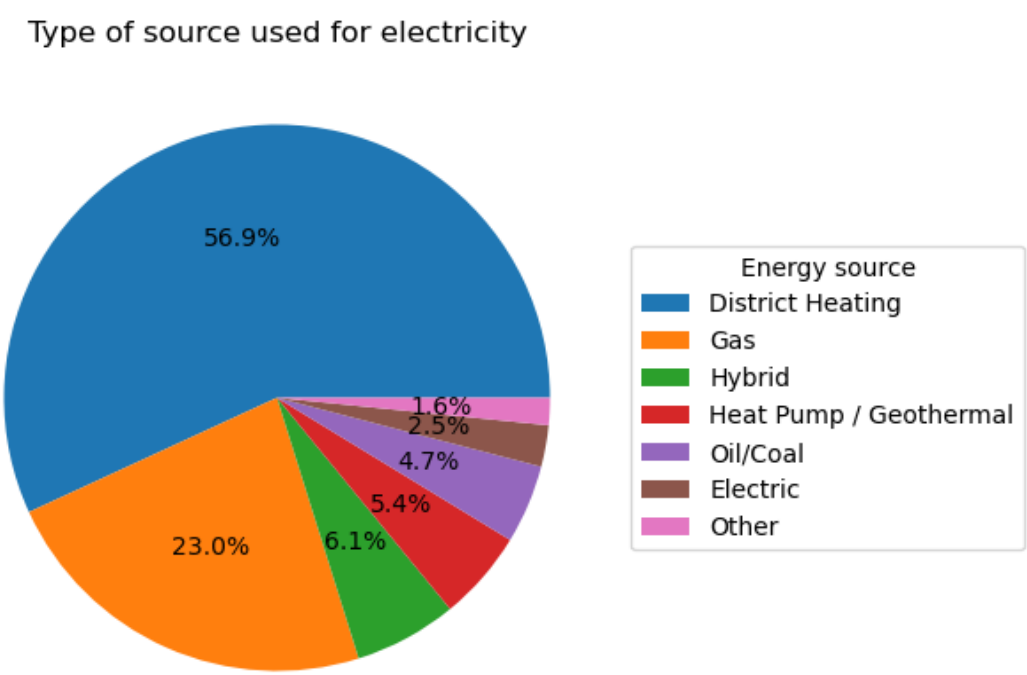
[16]:
```python
# drop duplicates
df = df.drop(index=[1738, 2069, 5494])

# for the rest change the values to 'Unknown' so we remove completely the␣
↪'Concrete' flooring type to avoid noise.
```

```
df.loc[[2058, 2555, 3356], 'flooring_type'] = 'unknown'
```

[17]:
```
counts = df["energy_source"].value_counts()

plt.figure(figsize=(5,5))

plt.pie(
    counts.values,
    autopct=lambda p: f'{p:.1f}%' if p > 1.5 else '',
    labeldistance=1.2
)

plt.legend(
    counts.index,
    title="Flooring Type",
    loc="center left",
    bbox_to_anchor=(1, 0.5)
)

plt.title("Type of source used for electricty")
plt.show()
```

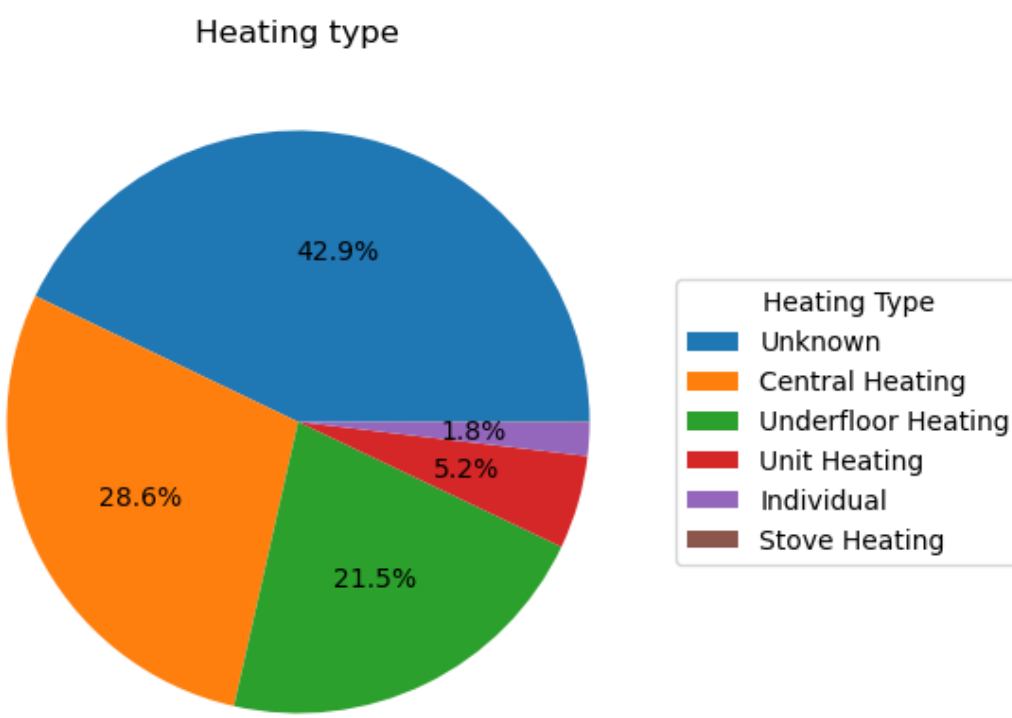


[18]:
```
# If we exclude unknown
counts = df[df["energy_source"] != "Unknown"]["energy_source"].value_counts()

plt.figure(figsize=(5,5))

plt.pie(
    counts.values,
    autopct=lambda p: f'{p:.1f}%' if p > 1.5 else '',
    labeldistance=1.2
)

plt.legend(
    counts.index,
    title="Energy source",
    loc="center left",
    bbox_to_anchor=(1, 0.5)
)

plt.title("Type of source used for electricity")
plt.show()
```



District Heating is the most popular and followed by gas. The rest are not very popular.

[19]:
```
counts = df["heating_type"].value_counts()

plt.figure(figsize=(5,5))

plt.pie(
    counts.values,
    autopct=lambda p: f'{p:.1f}%' if p > 1.5 else '',
    labeldistance=1.2
)

plt.legend(
    counts.index,
    title="Heating Type",
    loc="center left",
    bbox_to_anchor=(1, 0.5)
)

plt.title("Heating type")
plt.show()
```



[20]:
```python
# Deeper look for the 'Stove Heating' type
df["heating_type"].value_counts()
```

[20]:
```
heating_type
Unknown                6704
Central Heating        4467
Underfloor Heating     3357
Unit Heating            810
Individual              288
Stove Heating             5
Name: count, dtype: int64
```

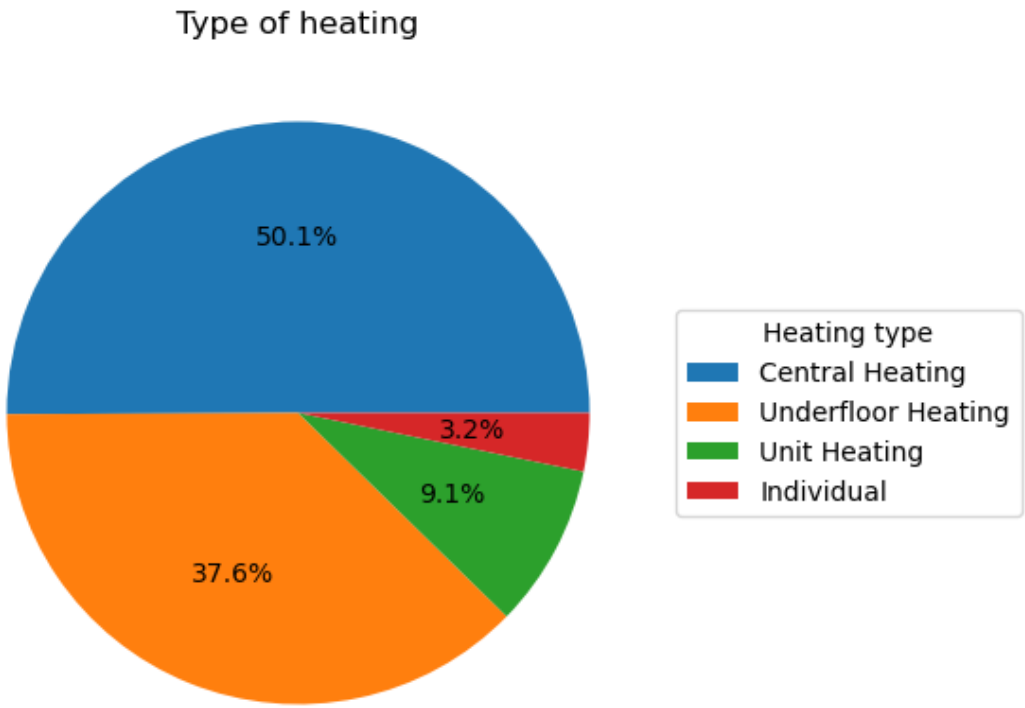
[21]:
```python
df[df["heating_type"] == "Stove Heating"]
```

[21]:
```
                                          listing_url  cold_rent  area  \
2175  https://www.immowelt.de/expose/30358959-c06c-4…    1172.83  69.0
3006  https://www.immowelt.de/expose/30358959-c06c-4…    1172.83  69.0
3881  https://www.immowelt.de/expose/30358959-c06c-4…    1172.83  69.0
4178  https://www.immowelt.de/expose/30358959-c06c-4…    1172.83  69.0
5904  https://www.immowelt.de/expose/2e141c4a-cfb4-4…    1172.83  69.0

      rooms  floor    free_from  has_balkon  has_terrasse  has_garten  \
2175    2.0    5.0  immediately       False         False       False
3006    2.0    4.0  immediately       False         False       False
3881    2.0    3.0  immediately       False         False       False
4178    2.0    2.0  immediately       False         False       False
5904    2.0    1.0  immediately        True         False       False

      elevator  …  is_barrier-free  has_built-in_kitchen  has_bathtub  \
2175     False  …            False                 False        False
3006     False  …            False                 False        False
3881     False  …            False                 False        False
4178     False  …            False                 False        False
5904     False  …            False                 False         True

      has_shower  flooring_type  energy_source   heating_type  \
2175       False        unknown        Unknown  Stove Heating
3006       False        unknown        Unknown  Stove Heating
3881       False        unknown        Unknown  Stove Heating
4178       False        unknown        Unknown  Stove Heating
5904       False           Wood        Unknown  Stove Heating

     property_condition year_built        district
2175   Needs Renovation        NaN  Friedrichshain
3006   Needs Renovation        NaN  Friedrichshain
3881   Needs Renovation        NaN  Friedrichshain
4178   Needs Renovation        NaN  Friedrichshain
```

```
5904          Renovated        NaN  Friedrichshain

[5 rows x 22 columns]
```

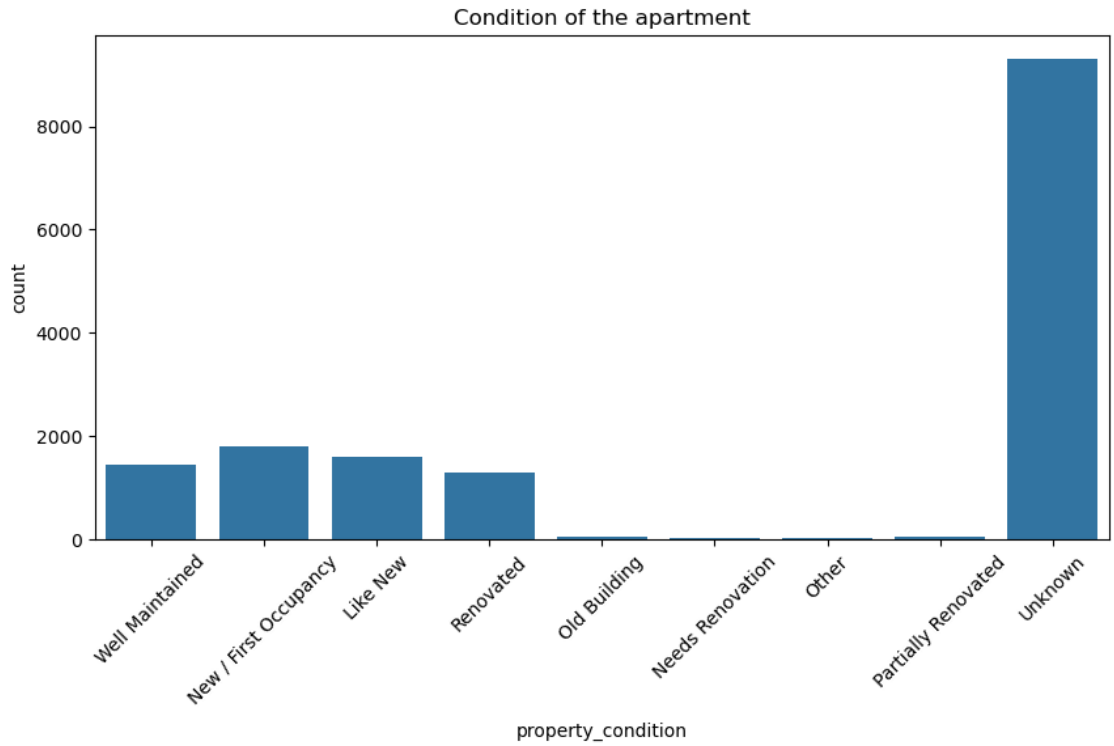
[22]:
```python
# These samples will just confuse our model so we just keep 2 and change their␣
↪heating_type to 'Unknown', this way we have
# one less category.

# drop duplicates
df = df.drop(index=[2175, 3006, 4178])

# for the rest change the values to 'Unknown'
df.loc[[3881, 5904], 'heating_type'] = 'Unknown'
```

[23]:
```python
# If we exclude unknown
counts = df[df["heating_type"] != "Unknown"]["heating_type"].value_counts()

plt.figure(figsize=(5,5))

plt.pie(
    counts.values,
    autopct=lambda p: f'{p:.1f}%' if p > 1.5 else '',
    labeldistance=1.2
)

plt.legend(
    counts.index,
    title="Heating type",
    loc="center left",
    bbox_to_anchor=(1, 0.5)
)

plt.title("Type of heating")
plt.show()
```



If we exclude the samples whose heating type is not known, we have the Central Heating as the most popluar form of heating with half of the apartments, followed by Underfloor heating with around one third. I didn't except Underfloor heating to be this popular since it is more expensive than the rest. Unit heating and Individual Heating are not as popular as the other two.

[24]:
```
fig, ax = plt.subplots(figsize=(10, 5))
sns.countplot(data=df, x="property_condition", ax=ax)

ax.xaxis.labelpad = 8
plt.xticks(rotation=45)
plt.title("Condition of the apartment")
plt.show()
```



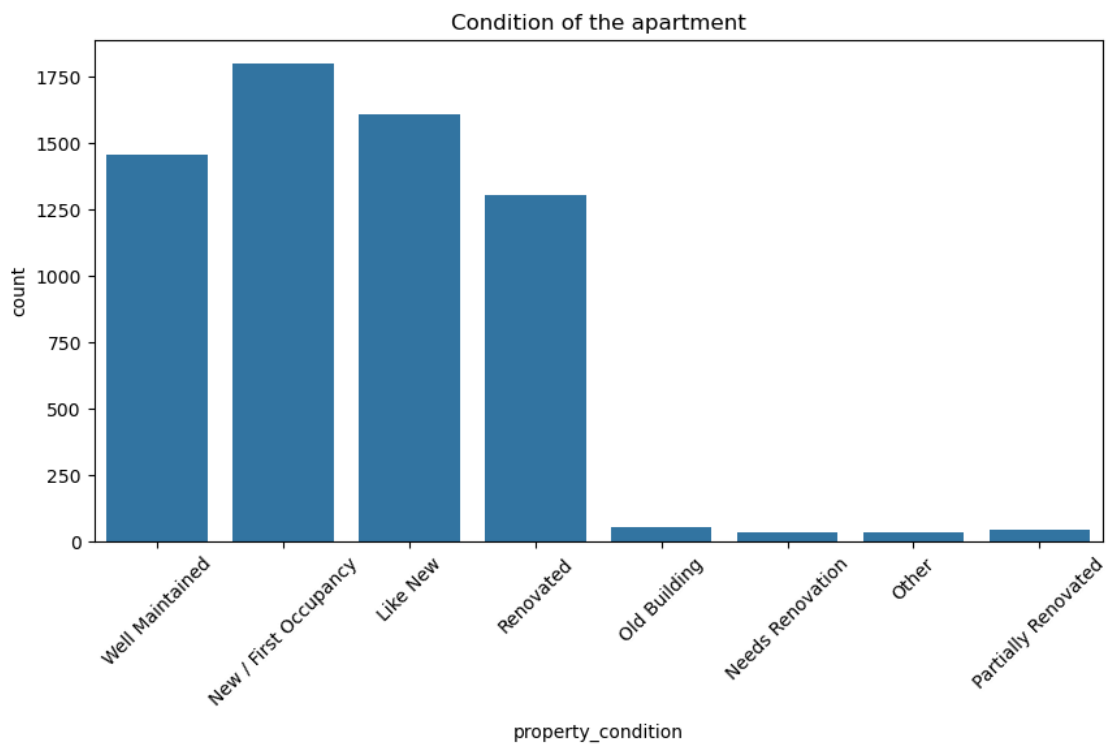
[25]:
```
df["property_condition"].value_counts()
```

[25]:
```
property_condition
Unknown                  9297
New / First Occupancy    1796
Like New                 1609
Well Maintained          1453
Renovated                1305
Old Building               56
Partially Renovated        44
Other                      35
Needs Renovation           33
Name: count, dtype: int64
```

Every category contains meaningful information so we cannot drop any of them, even though they occur in very few samples compared to the rest of categories. Some of categories might seem very similar to each-other, but they carry slightly different meaning so we cannot merge them without losing information. Also we don't have many categories so we don't have the pressure of merging them.

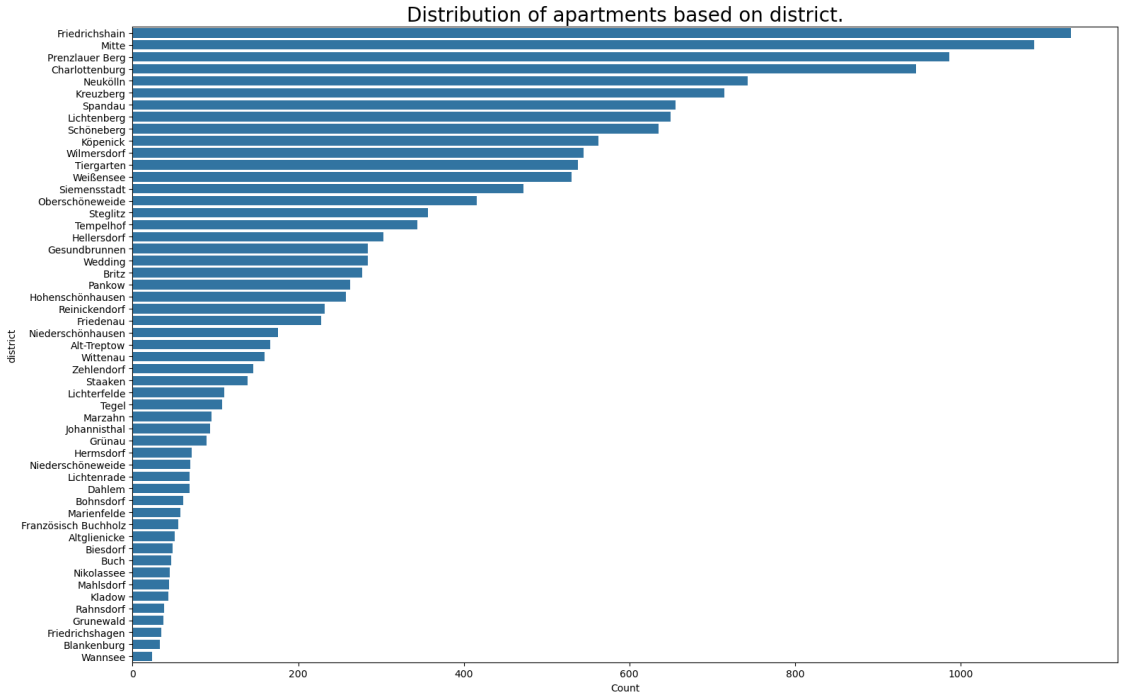
[26]:
```
# If we exclude unknown
fig, ax = plt.subplots(figsize=(10, 5))
```

```
sns.countplot(data=df[df["property_condition"] != "Unknown"],␣
↪x="property_condition")
plt.title("Condition of the apartment")
ax.xaxis.labelpad = 8
plt.xticks(rotation=45)
plt.show()
```



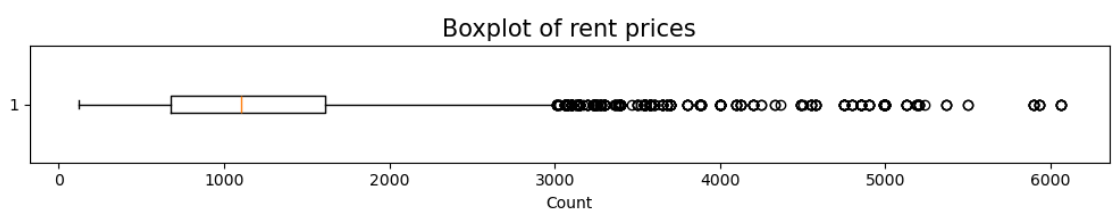
We notice a huge difference between the first 4 categories which occur very often and the second 4 categories which occur very rarely. The first 4 categories make up most of the apartments (if we exclude unknown) and the other 4 which are associated with negative meaning make up only a tiny portion of the apartments. Looking at this graph we might get confused and conclude that alomst every apartment in Berlin is in a very good condition, which is not true because we are leaving out the unknown category. We can definitely hypothise that 'Unknown' category will have some negative impact to the price and condition of the apartment. We might reveal this information later when we do the Bivariate Analysis or if our model pick up these patterns.

[27]:
```
plt.figure(figsize=(16, 10))
sns.countplot(data=df, y="district", order=df["district"].value_counts().index)
plt.title('Distribution of apartments based on district.',size=20)
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```



The most popular area is Friedrichshain followed by Mitte, Prenzlauer Berg, Charlottenburg, Neukölln, Kreuzberg and so on.

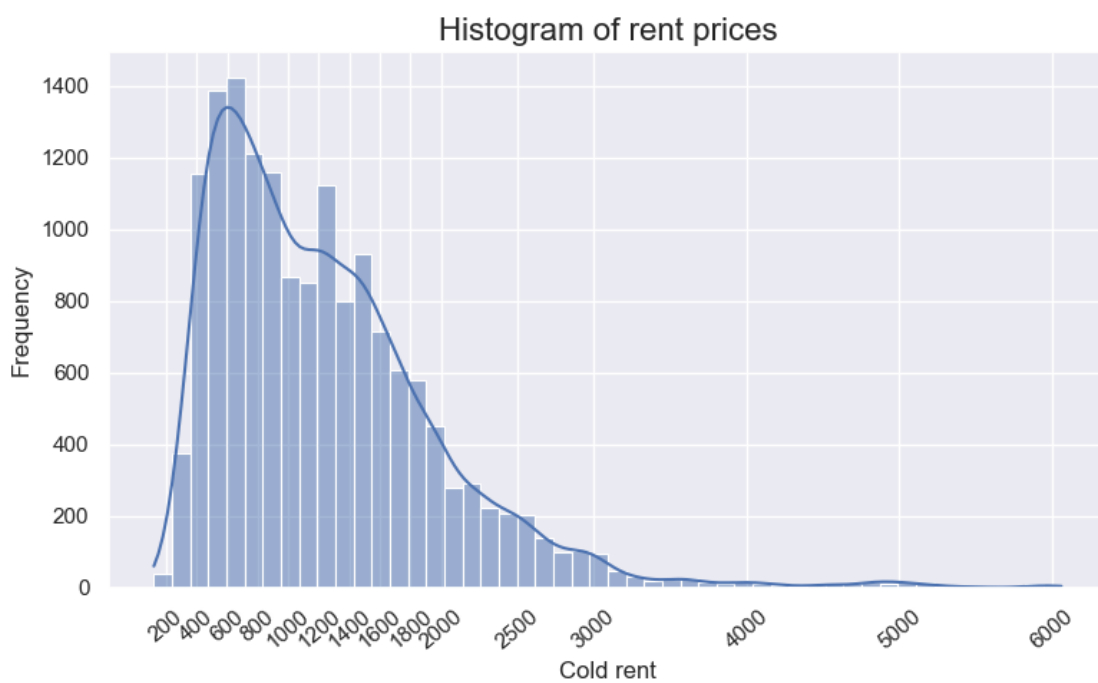
[28]:
```
plt.figure(figsize=(10, 2))
plt.boxplot(df["cold_rent"],vert=False)
plt.title('Boxplot of rent prices',size=15)
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```



We already dropped the top 1% of values as outliers and decided to keep everything below 6000 euros. I wouldn't say all those points are outliers and should not be dropped.

[29]:
```
plt.figure(figsize=(8, 5))
sns.set(style="darkgrid")
```

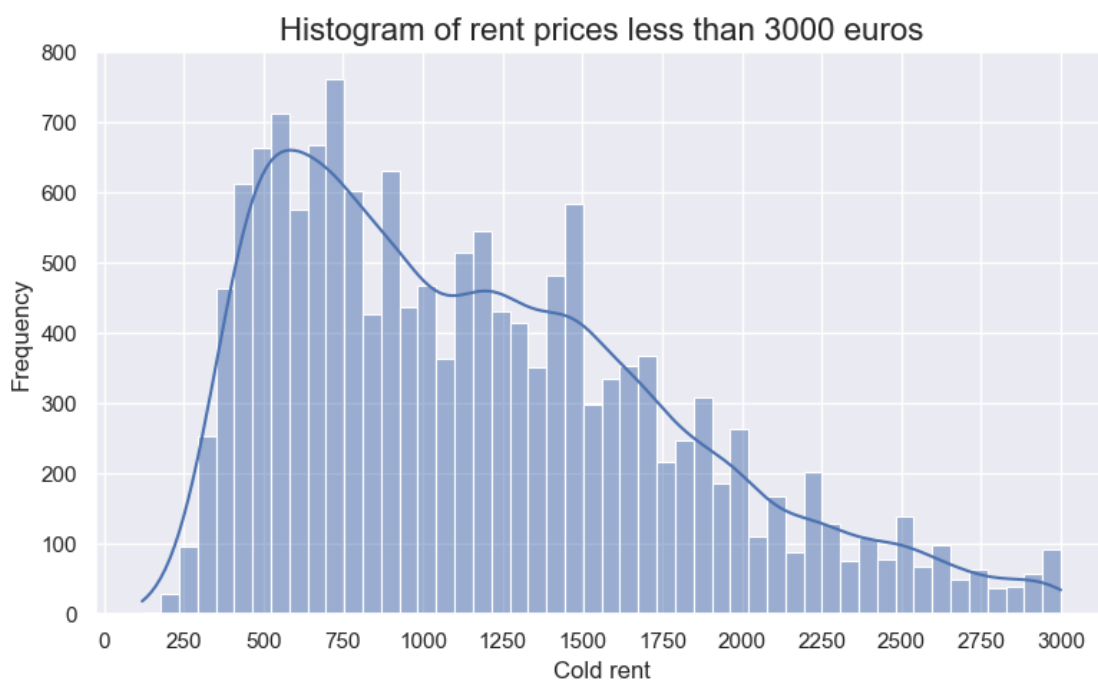
```
ax = sns.histplot(
    df["cold_rent"],
    bins=50,
    kde=True,
)

ticks = [200,400,600,800,
         1000,1200,1400,1600,1800,2000,2500,
         3000,4000,5000,6000]

ax.set_xticks(ticks)
plt.xticks(rotation=40)

plt.title('Histogram of rent prices', size=16)
plt.xlabel('Cold rent')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



The distribution of rent prices has a high positive skewness and is mostly focused between 500 and 2000.

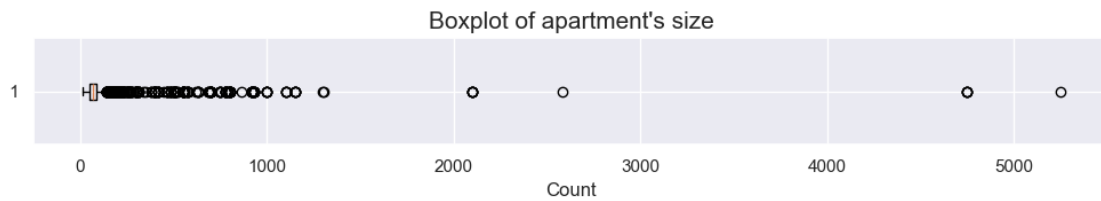
[30]:
```
plt.figure(figsize=(8, 5))
sns.set(style="darkgrid")
```

```
plot_df = df[df["cold_rent"] <= 3000]

ax = sns.histplot(
    plot_df["cold_rent"],
    bins=50,
    kde=True
)

ticks = np.linspace(0,3000,13)

ax.set_xticks(ticks)

plt.title('Histogram of rent prices less than 3000 euros', size=16)
plt.xlabel('Cold rent')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



The most common prices is around 750 euros and with most of the samples distributed around 600 euros. Inside the range of 1000-1500 there is also a huge cluster of samples distributed. We can notice some spkes at certain values such as 750, 1200 and 1500.

[31]:
```
plt.figure(figsize=(10, 2))
plt.boxplot(df["area"],vert=False)
plt.title("Boxplot of apartment's size",size=15)
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```

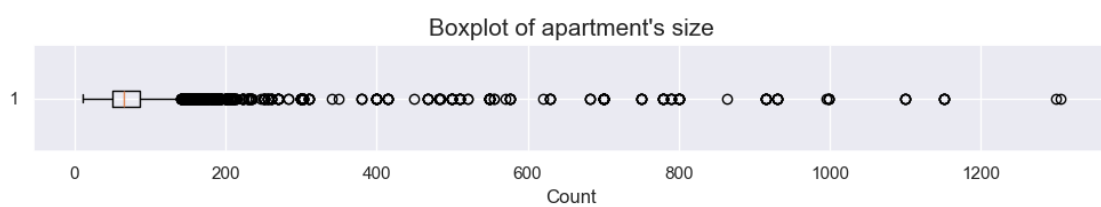


[32]:
```
len(df[df["area"] > 2000])
```

[32]: 11

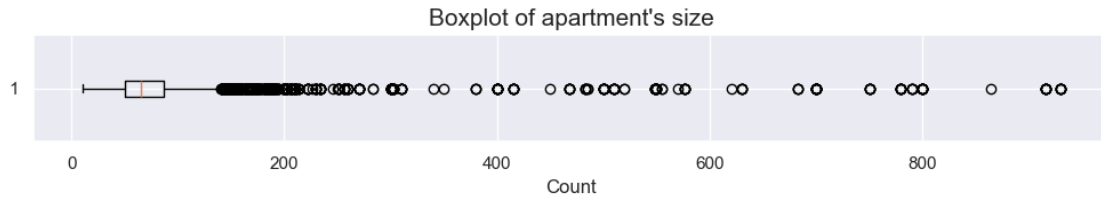
[33]:
```
# 11 values out 15500 should definitely treated as outliers. This is clearly␣
↪shown by the boxplot we just plotted.
# They only harm out model so they should be dropped.
df = df[df["area"] < 2000]
```

[34]:
```
plt.figure(figsize=(10, 2))
plt.boxplot(df["area"],vert=False)
plt.title("Boxplot of apartment's size",size=15)
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```

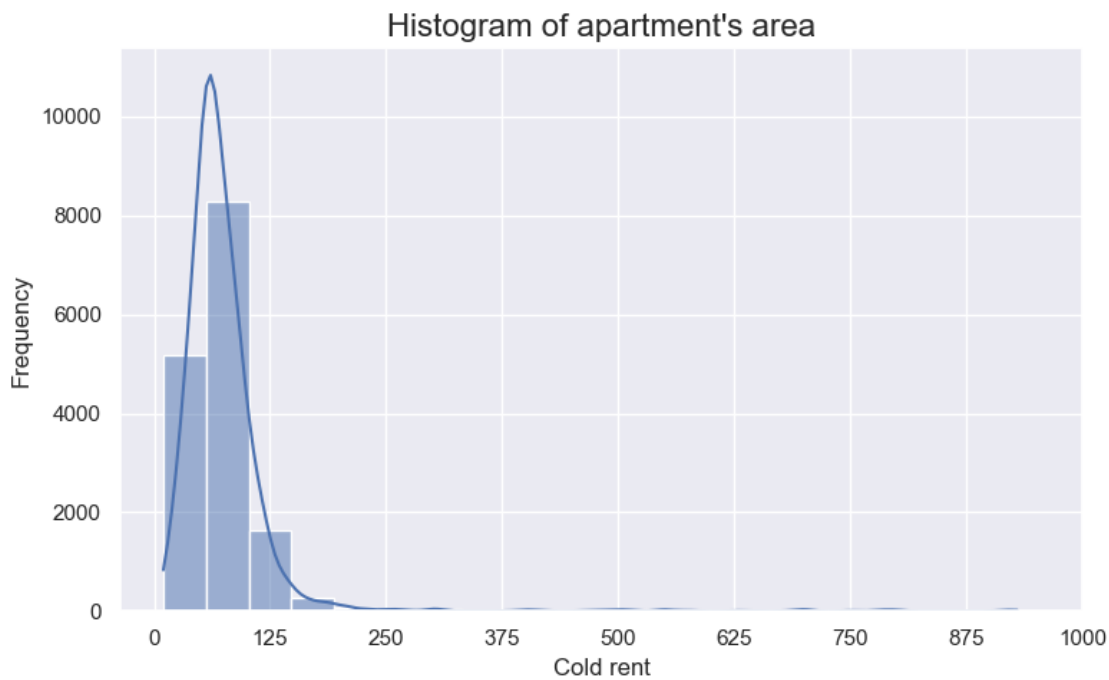


[35]:
```
# Those dots from starting from 1000 also seem as outliers that harm our models␣
↪so let's remove them
df = df[df["area"] < 950]

plt.figure(figsize=(10, 2))
plt.boxplot(df["area"],vert=False)
```

```
plt.title("Boxplot of apartment's size",size=15)
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```



[36]:
```
# Let's also plot the histogram and distribution of values to get more␣
↪information
plt.figure(figsize=(8, 5))
sns.set(style="darkgrid")

ax = sns.histplot(
    df["area"],
    bins=20,
    kde=True
)

ticks = np.linspace(0,1000,9)

ax.set_xticks(ticks)

plt.title("Histogram of apartment's area", size=16)
plt.xlabel('Cold rent')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```

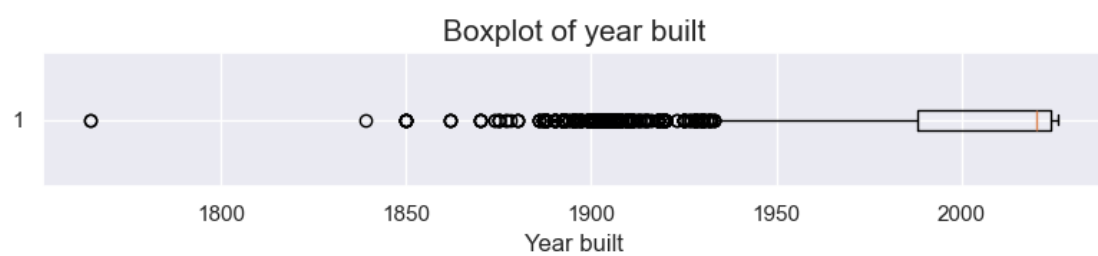


The distribution has a very long right tail and most values are distributed around 50 square meters.

[37]:
```
plt.figure(figsize=(8, 5))
sns.set(style="darkgrid")

ax = sns.histplot(
    df[df["area"]<=200]["area"],
    bins=20,
    kde=True
)

ticks = np.linspace(0,200,9)

ax.set_xticks(ticks)

plt.title("Histogram of apartments' area (range [0-200])", size=16)
plt.xlabel('Cold rent')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```

It looks normally distributed around this common area range.

[38]:
```
plt.figure(figsize=(8, 2))

plt.boxplot(df["year_built"].dropna(), vert=False)

plt.title("Boxplot of year built", size=15)
plt.xlabel("Year built")

plt.tight_layout()
plt.show()
```

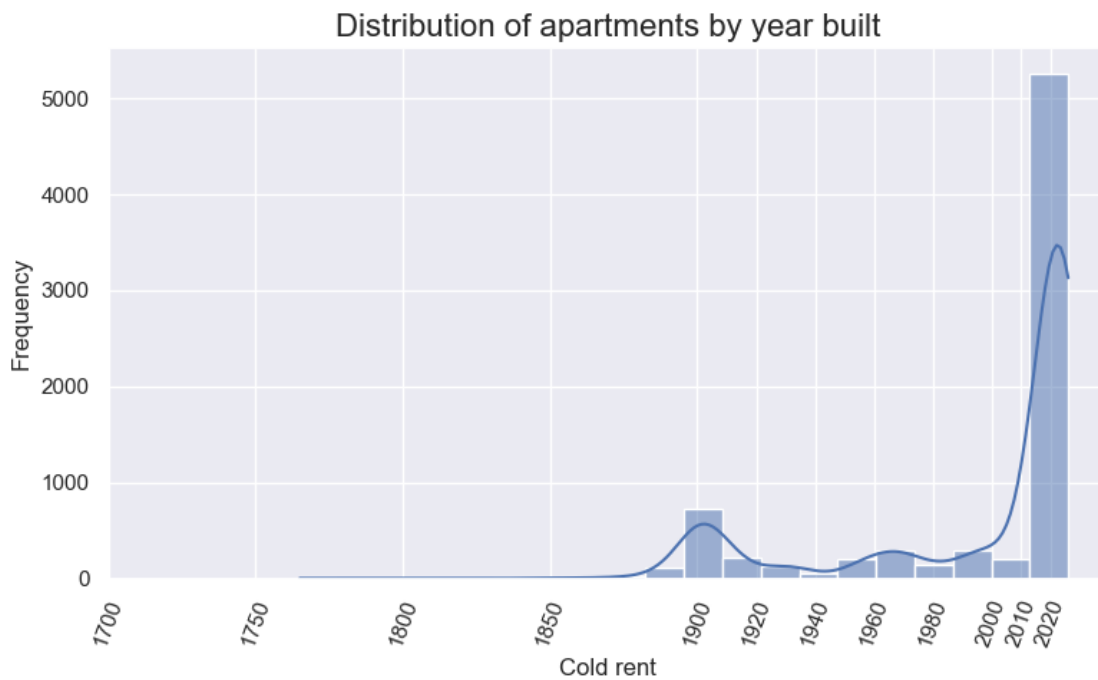


[39]:
```
df["year_built"].describe()
```

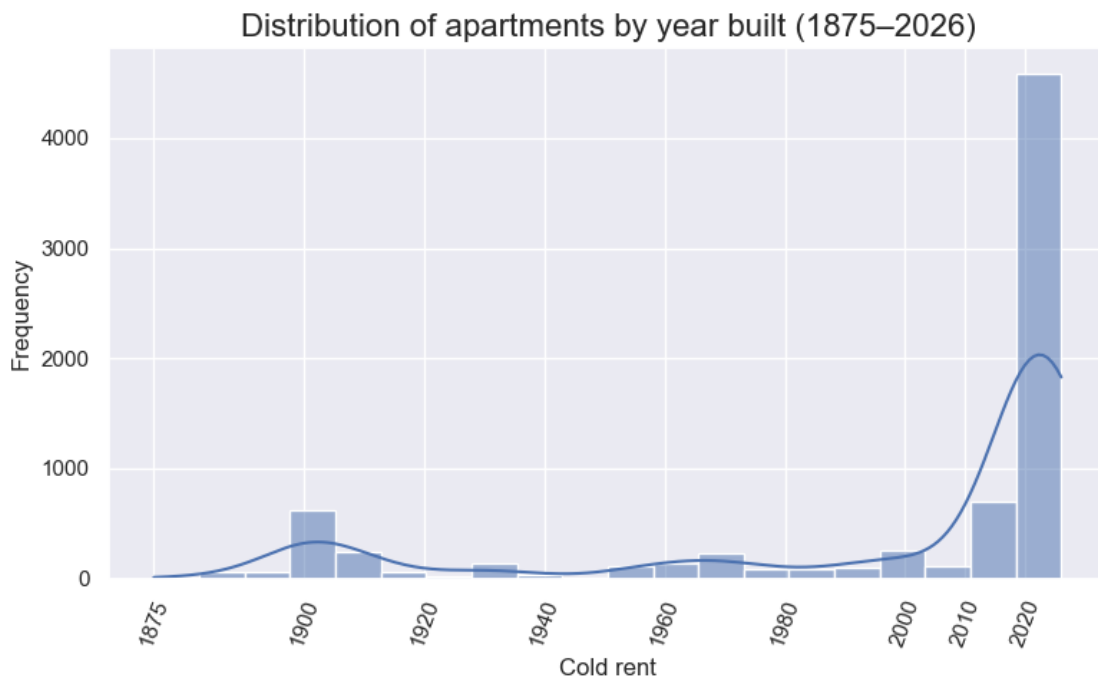
[39]:
```
count    7662.000000
mean     1996.980684
std        43.904575
min      1765.000000
25%      1988.000000
50%      2020.000000
75%      2024.000000
max      2026.000000
Name: year_built, dtype: float64
```

It is pretty interesting how we have only one apartment way older than others, built in year 1765. From this boxplot is not very easy to get a lot of information so let's also plot the distribution of values.

[40]:
```
plt.figure(figsize=(8, 5))
sns.set(style="darkgrid")

ax = sns.histplot(
    df["year_built"].dropna(),
    bins=20,
    kde=True
)

ticks = [1700,1750,1800,1850,1900,1920,1940,1960,1980,2000,2010,2020]

ax.set_xticks(ticks)

plt.title("Distribution of apartments by year built", size=16)
plt.xlabel('Cold rent')
plt.ylabel('Frequency')
plt.xticks(rotation=70)
plt.tight_layout()
plt.show()
```



[41]:
```
plt.figure(figsize=(8, 5))
sns.set(style="darkgrid")

ax = sns.histplot(
    df[df["year_built"] >= 1875]["year_built"],
    bins=20,
    kde=True
)

ticks = [1875,1900,1920,1940,1960,1980,2000,2010,2020]

ax.set_xticks(ticks)

plt.title("Distribution of apartments by year built (1875-2026)", size=16)
plt.xlabel('Cold rent')
plt.ylabel('Frequency')
plt.xticks(rotation=70)
plt.tight_layout()
plt.show()
```



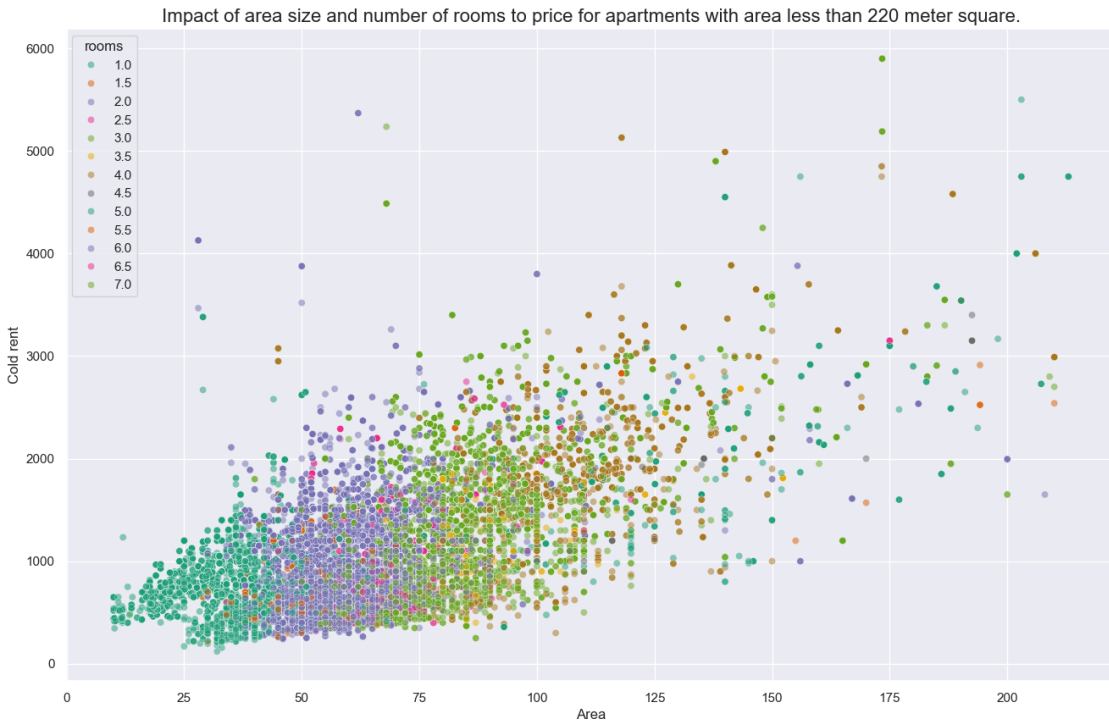
It is fascinating to see how both World Wars clearly caused the number of apartments built to drop. Also most of the apartments being rent out are built during the last 7 years. That makes sense considering the fact that old apartments are taken by locals.

Part 2: Understanding relationships between variables

[42]:
```
plt.figure(figsize=(16, 10))
plt.title("Impact of area size and number of rooms to price", size=16)
plt.xlabel('Area')
plt.ylabel('Cold rent')
sns.
 ↪scatterplot(x="area",y="cold_rent",data=df,hue="rooms",palette="Dark2",alpha=0.
 ↪5)
plt.show()
```

As expected a positive correlation, we have a very strong positive correlation, especially if we exclude apartments with areas larger than 200 meters. Similar trend is seen with the number of rooms, bigger area would mean a large number of rooms.

[43]:
```
plt.figure(figsize=(16, 10))
plt.title("Impact of area size and number of rooms to price for apartments with␣
↪area less than 220 meter square.", size=16)
plt.xlabel('Area')
plt.ylabel('Cold rent')
sns.scatterplot(x="area",y="cold_rent",data=df[df["area"] <=␣
↪220],hue="rooms",palette="Dark2",alpha=0.5)
plt.show()
```



[44]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(x="cold_rent", hue="floor", data=df)
plt.title("Impact of apartment floor on the price", size=16)
plt.xlabel("Cold rent")
plt.ylabel("Floor")
plt.show()
```

It is not very easy to tell from with 100% confidence from the graph wether the floor affacts the price or not. The best way would be to make a statistical test, Kruskal–Wallis test to be more specific. This test is more appropirate over One-way ANOVA because we have unparametric data. H0: The population medians of cold rent are equal across floors H1: At least one group differs

[45]:
```
from scipy.stats import kruskal

groups = [group["cold_rent"].values
          for name, group in df.groupby("floor")]

stat, p = kruskal(*groups)

print("Statistic:", stat)
print("p-value:", p)
```

```
Statistic: 905.7349306190807
p-value: 3.3701091987383317e-186
```

For $p < 0.05$ we reject the null hypothesis, therefore we can say that there is clearly a difference between the price distribution for different floors.

We could argue that the result for that might be the one floor category that is visibly skewed more left than the others. Let's redo the test by excluding that category.

[46]:
```
df_test = df[df["floor"] != 0]
```

[47]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(x="cold_rent", hue="floor", data=df_test)
plt.title("Impact of apartment floor on the price (without ground floor)",␣
↪size=16)
plt.xlabel("Cold rent")
plt.ylabel("Floor")
plt.show()
```

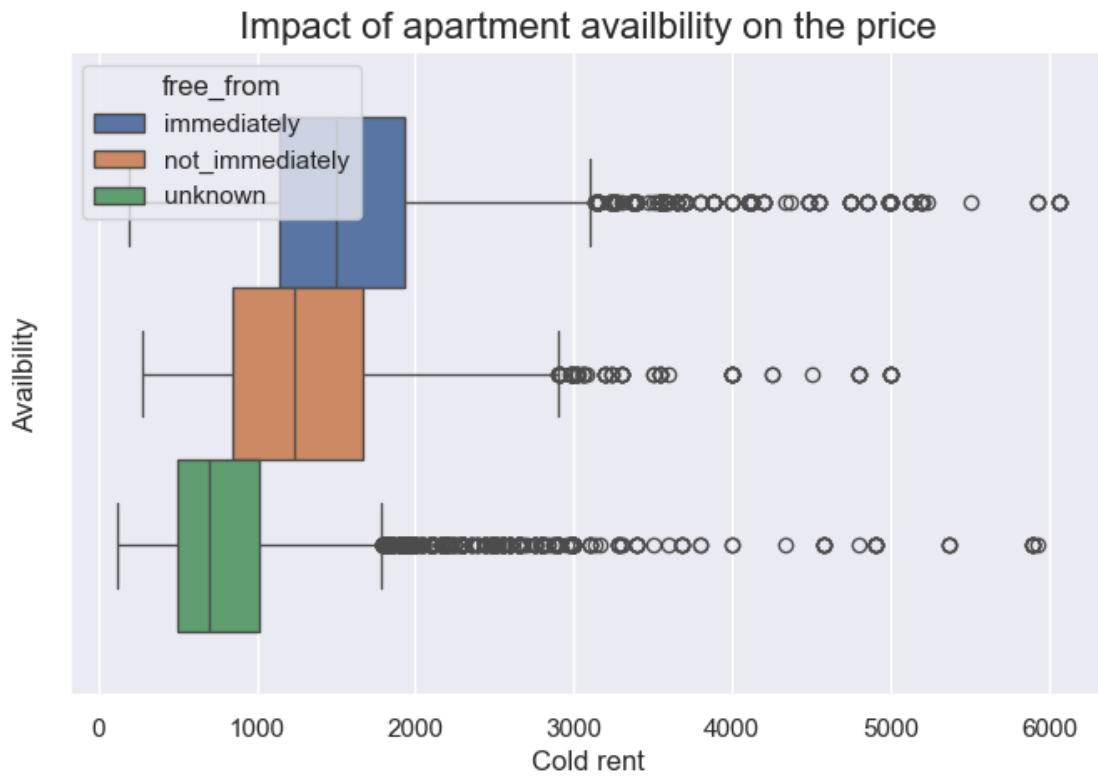
[48]:
```
groups = [group["cold_rent"].values
          for name, group in df_test.groupby("floor")]

stat, p = kruskal(*groups)

print("Statistic:", stat)
print("p-value:", p)
```

```
Statistic: 133.53372217126622
p-value: 3.349692120836482e-23
```

We reject the null hypothesis hypothesis again, meaning they still have different distributions. This time we can clearly notice the impact of the ground floor category by observing the huge increase

in the value of p.

[49]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(x="cold_rent", hue="free_from", data=df)
plt.title("Impact of apartment availbility on the price", size=16)
plt.xlabel("Cold rent")
plt.ylabel("Availbility")
plt.show()
```

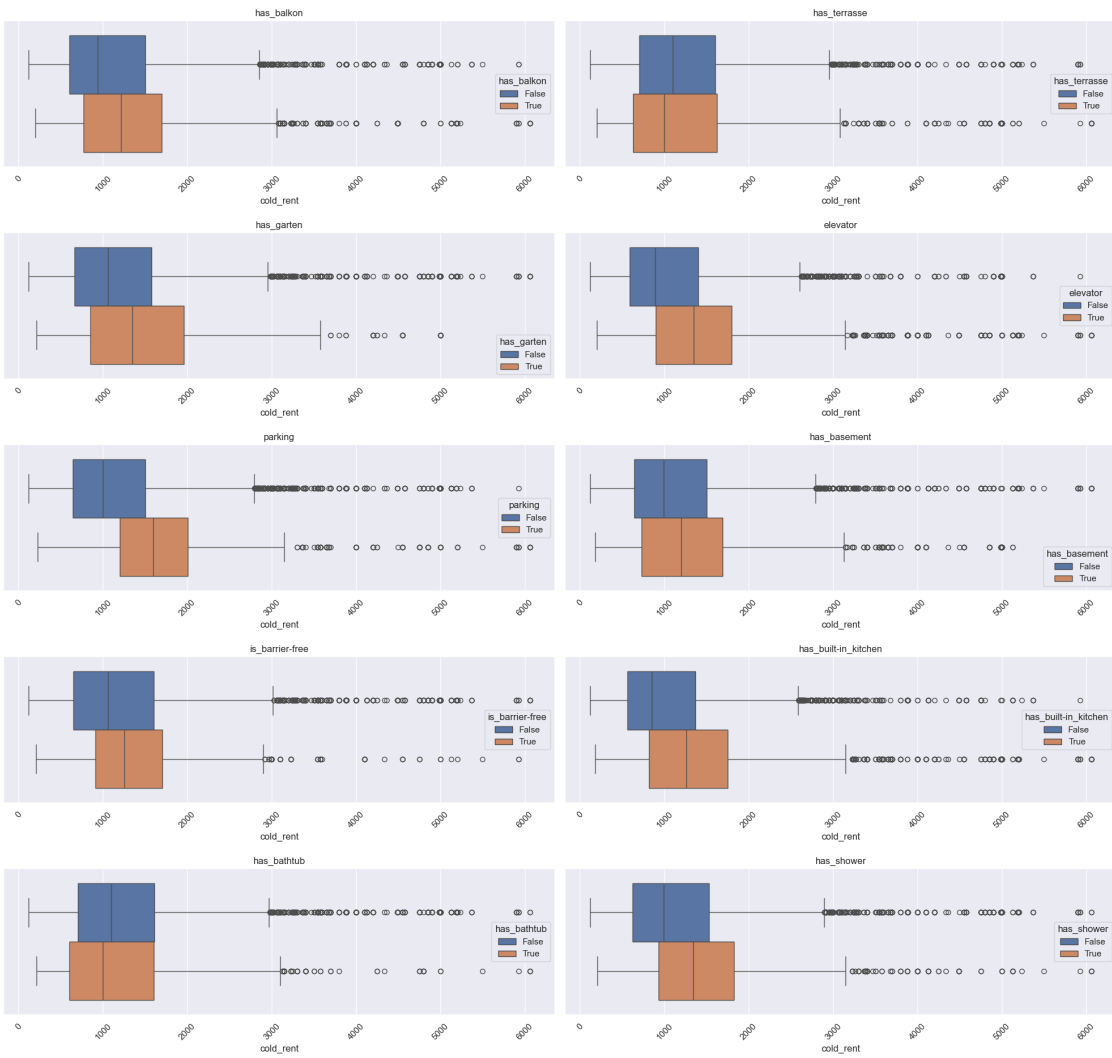


The imapct here is noticable and turn out to be as expeceted. Apartments that are available immediately are more likely to be more expecive compare to those that require a waiting time before moving in. When there is an uncertainty on the availbility the price is expected to be even lower.

[50]:
```
features = ['has_balkon', 'has_terrasse',␣
↪'has_garten','elevator','parking','has_basement','is_barrier-free','has_built-in_kitchen','

fig, axes = plt.subplots(nrows=5, ncols=2, figsize=(20, 20))
fig.suptitle("Distribution of prices based on binary categorical features.
↪",size=30)

for ax, feature in zip(axes.flatten(), features):
```

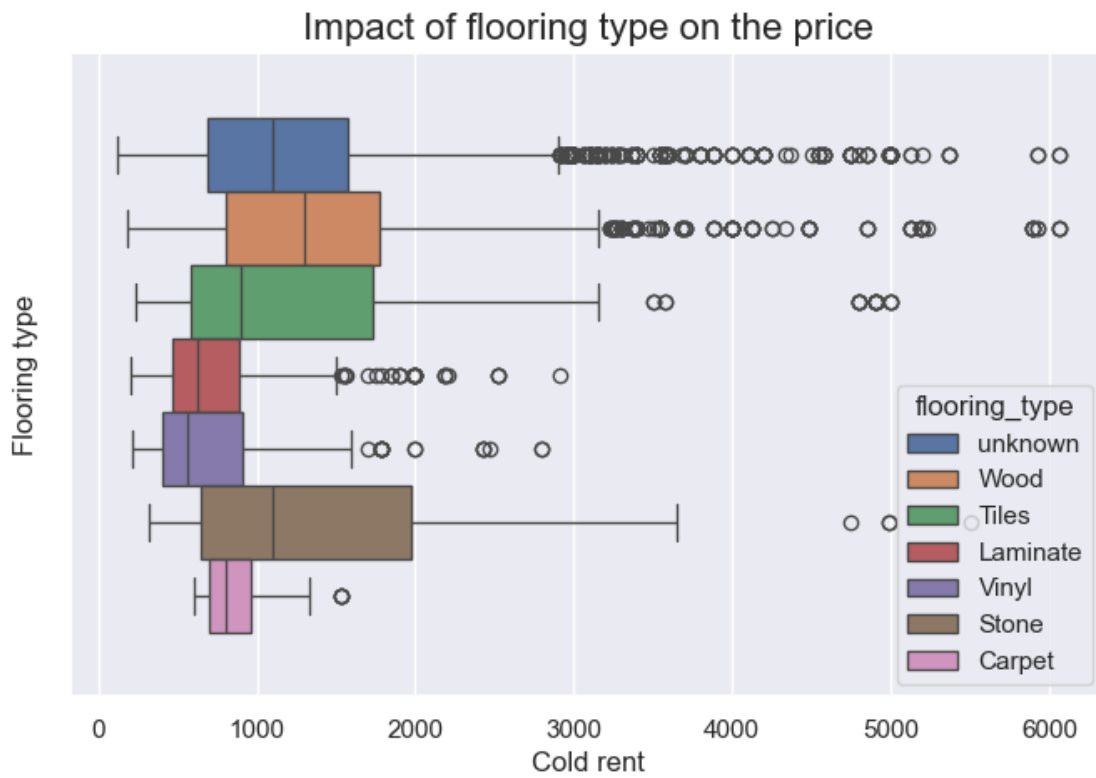
```
    sns.boxplot(data=df, x="cold_rent", hue=feature, ax=ax)
    ax.set_title(feature)
    ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.subplots_adjust(hspace=0.45, top=0.92)
plt.show()
```

Distribution of prices based on binary categorical features.



We can notice that if an apartment poses a feature it tends to be more expensive, with some of the features having a slightly stronger effect than the others, for examples having a shower, a park, an elevator or a built-in kitchen makes a greater differece compared to the other features.

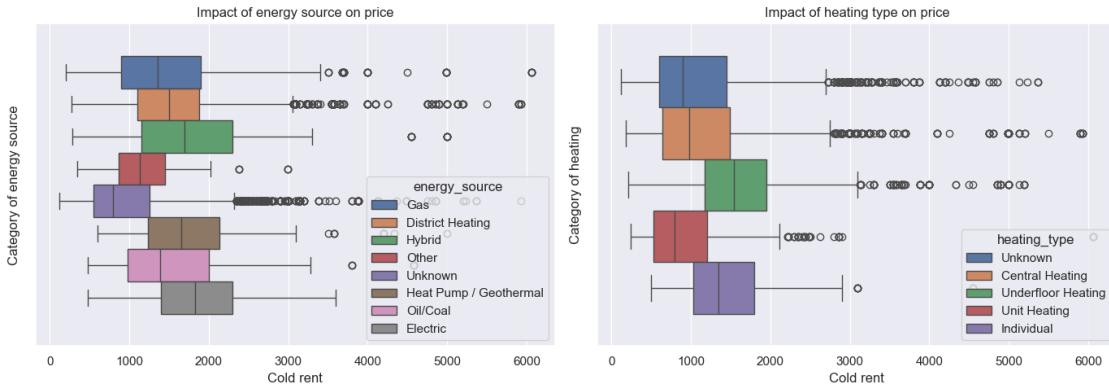
[51]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(x="cold_rent", hue="flooring_type", data=df)
plt.title("Impact of flooring type on the price", size=16)
plt.xlabel("Cold rent")
plt.ylabel("Flooring type")
plt.show()
```



This feature turns out to be economically meaningful, with certain good materials such as wood, stone and tiles being associated with higher rent compared to materials like carpet, vinyl and laminate which correspont to lower rent prices.

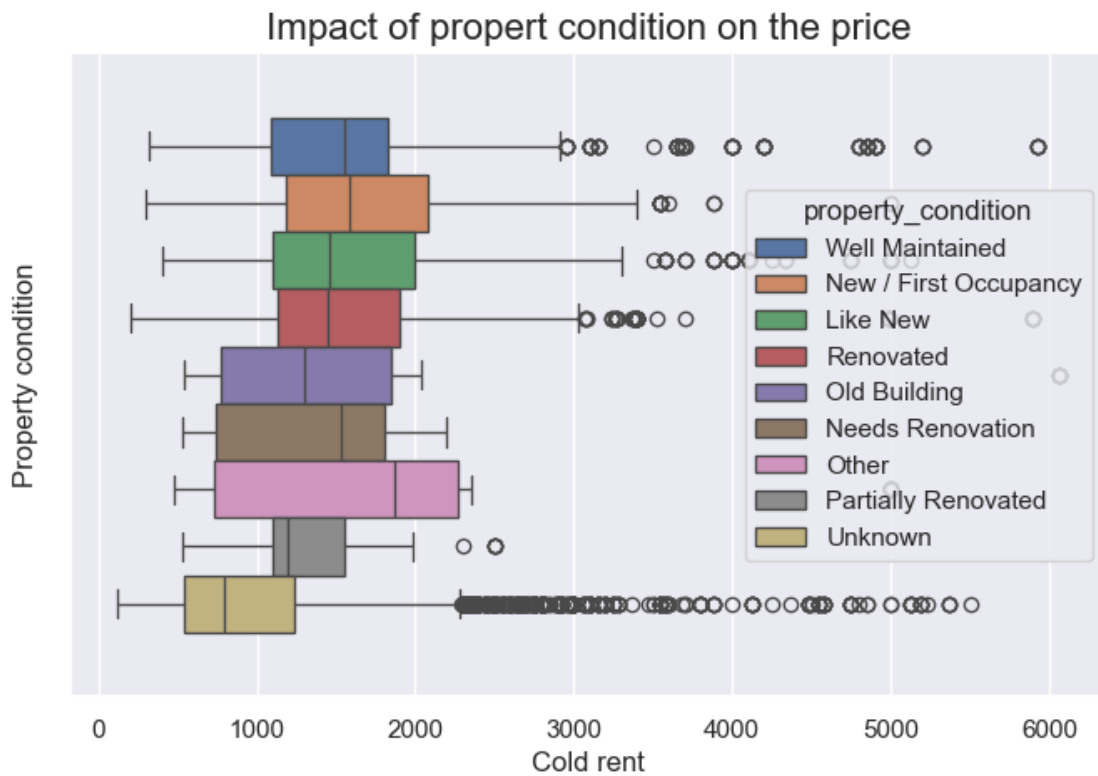
[52]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(x="cold_rent", hue="energy_source", data=df, ax=axes[0])
axes[0].set_title("Impact of energy source on price")
axes[0].set_xlabel("Cold rent")
axes[0].set_ylabel("Category of energy source")

sns.boxplot(x="cold_rent", hue="heating_type", data=df, ax=axes[1])
axes[1].set_title("Impact of heating type on price")
axes[1].set_xlabel("Cold rent")
axes[1].set_ylabel("Category of heating")
```

```
plt.tight_layout()
plt.show()
```



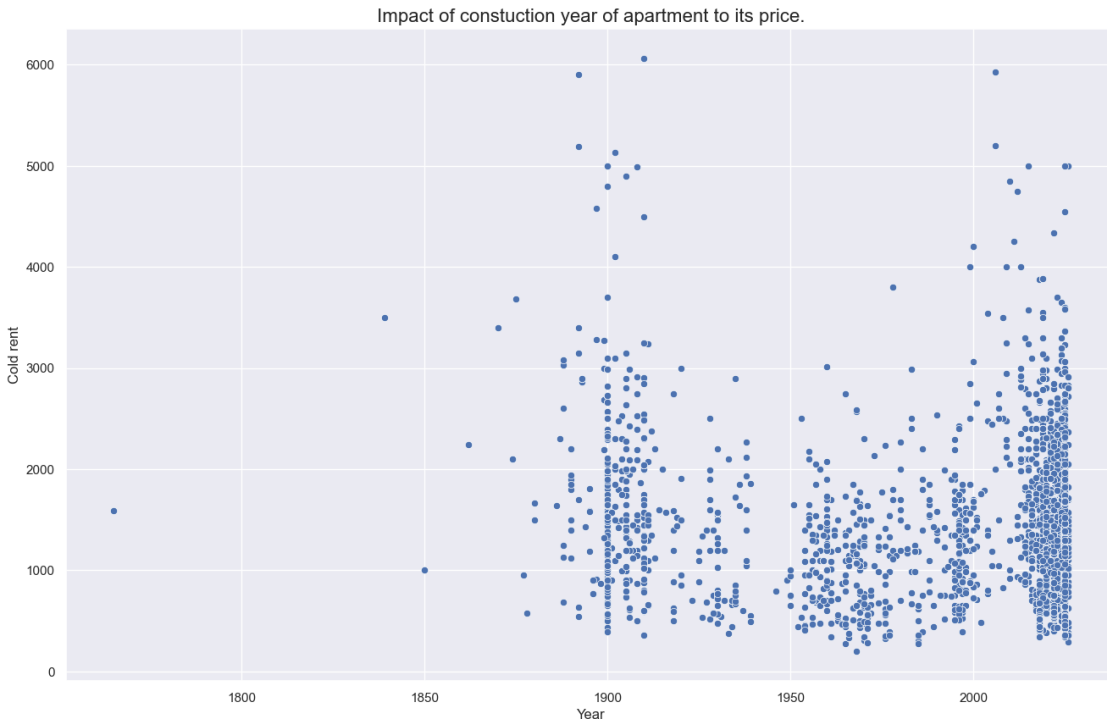
On the left graph we see the distribution of rent prices based on different energy sources. We notice that when we don't have information about the energy sources used we tend to have lower prices. Unlike other categories this time not knowing the category is associated with a strong negative effect. Features like Hybrid and Eletric are assosiated with higher prices of rent compared to the others, but overall they tend to have a relatively similar distribution if we exclude the "Unknown" and "Other" features. On the right graph we see the distribution of rent prices based on different heating types. As expected, underfloor heating is associated with higher rent prices. Also individual heating tends to have higher prices similar to underfloor heaing, while the rest of categories having a lower distribution with the unit heating having the lowest.

[53]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(x="cold_rent", hue="property_condition", data=df)
plt.title("Impact of propert condition on the price", size=16)
plt.xlabel("Cold rent")
plt.ylabel("Property condition")
plt.show()
```



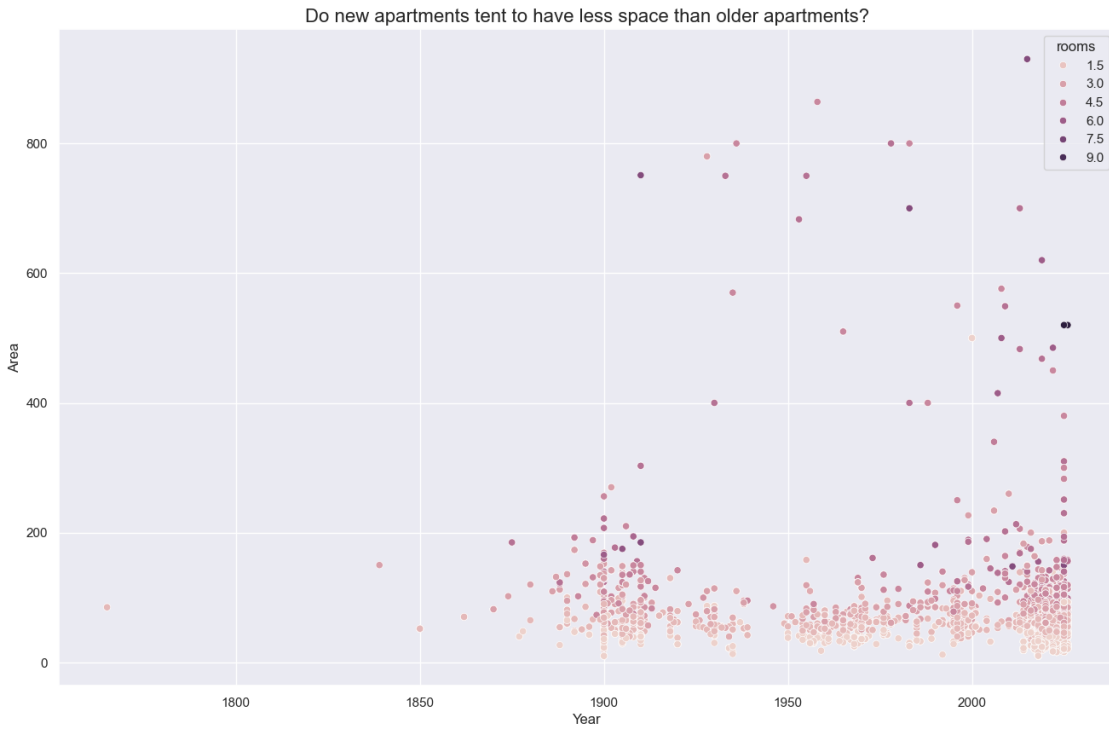
Unlike most of the features, this time the results seem to be slightly economically unsatisfactory, because we notice feautures like "Needs Renovation" to be assosiated on avarage with higher prices than apartments which are on a good condition. But when we compare the categories associated with a positive impact, they tend to be associated with higher prices than features like "Partially Renovated", "Old Building" or when the property is not mentioned at all it tends to be associated with even lower prices.

[54]:
```
plt.figure(figsize=(16, 10))
plt.title("Impact of constuction year of apartment to its price.", size=16)
plt.xlabel('Year')
plt.ylabel('Cold rent')
sns.scatterplot(x="year_built",y="cold_rent",data=df)
plt.show()
```



It seems like there is no clear effect on the apartment's price. We can notice that there is a higher diversity of prices after year 2000 and around year 1900. This could be associated with economical progress of Germany and Berlin during those times.

[55]:
```
plt.figure(figsize=(16, 10))
plt.title("Do new apartments tent to have less space than older apartments?",␣
 ↪size=16)
plt.xlabel('Year')
plt.ylabel('Area')
sns.scatterplot(x="year_built",y="area",data=df,hue="rooms")
plt.show()
```



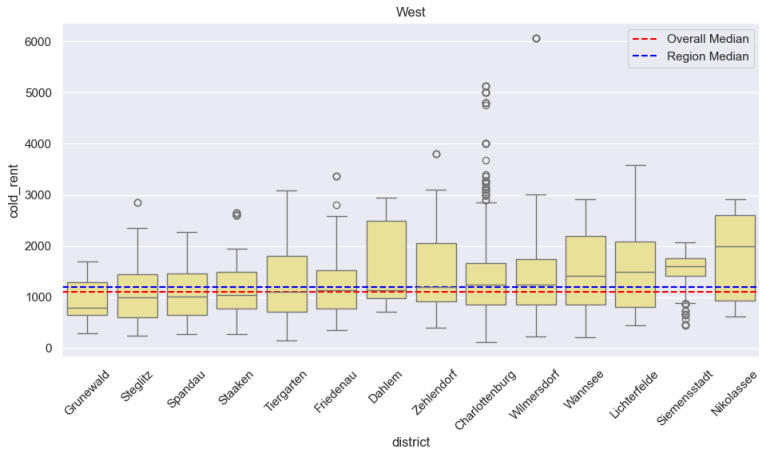
We can notice the same result, that there is a higher diversity of aparments around year 1900 and after year 2000.

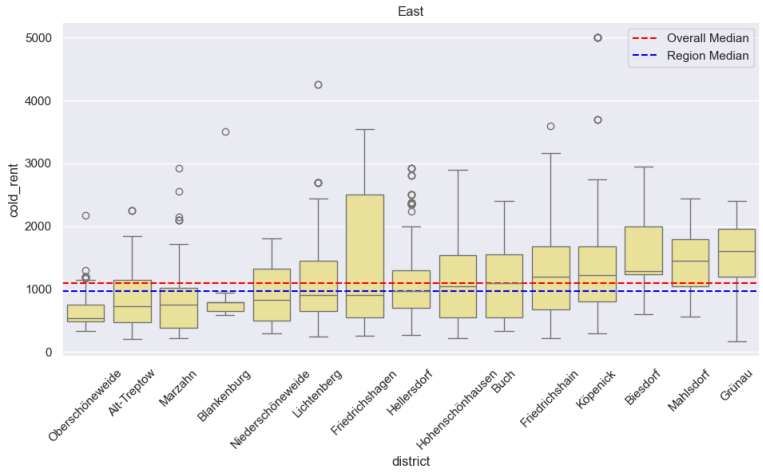
[56]:
```
west_berlin = ["Charlottenburg", "Wilmersdorf", "Spandau", "Zehlendorf",␣
 ↪"Dahlem", "Grunewald", "Wannsee", "Steglitz",
                "Lichterfelde", "Friedenau",␣
 ↪"Siemensstadt","Staaken","Nikolassee","Tiergarten"]
east_berlin = ["Friedrichshain", "Lichtenberg", "Hohenschönhausen", "Marzahn",␣
 ↪"Hellersdorf", "Köpenick", "Friedrichshagen",
               "Oberschöneweide", "Niederschöneweide", "Alt-Treptow", "Grünau",␣
 ↪"Mahlsdorf", "Biesdorf", "Blankenburg", "Buch"]
north_berlin = ["Pankow", "Weißensee", "Niederschönhausen", "Hermsdorf",␣
 ↪"Tegel", "Wittenau", "Reinickendorf", "Gesundbrunnen",
               "Wedding","Prenzlauer Berg","Französisch Buchholz"]
south_central_berlin = ["Mitte", "Neukölln", "Kreuzberg","Schöneberg",␣
 ↪"Tempelhof", "Britz", "Lichtenrade", "Marienfelde",
                       "Johannisthal", "Altglienicke", "Rahnsdorf", "Kladow",␣
 ↪"Bohnsdorf"]
region_map = {
    "West": west_berlin,
    "East": east_berlin,
    "North": north_berlin,
    "South/Central": south_central_berlin
```

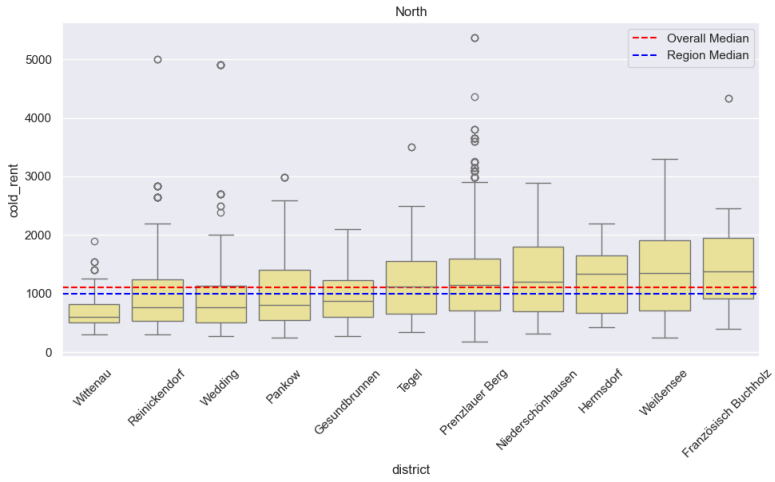
```
}
```

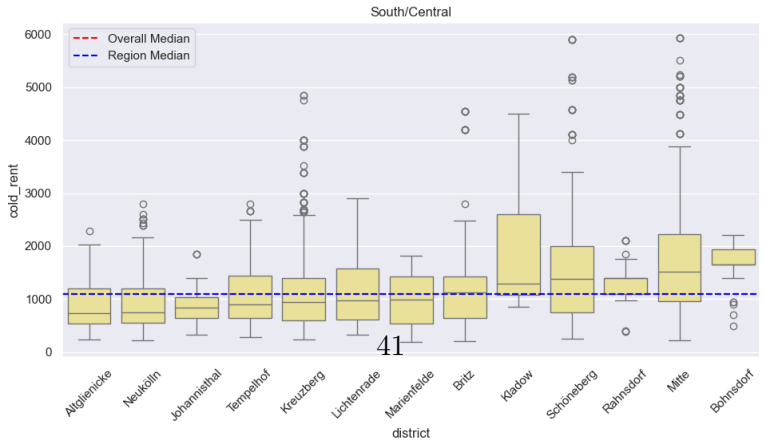
[58]:
```
fig, axes = plt.subplots(4, 1, figsize=(10, 24))

regions = list(region_map.keys())

overall_median = df["cold_rent"].median()

for ax, region in zip(axes, regions):

    subset = df[df["district"].isin(region_map[region])]

    median_order = (
        subset.groupby("district")["cold_rent"]
        .median()
        .sort_values()
        .index
    )

    # Boxplot
    sns.boxplot(
        data=subset,
        x="district",
        y="cold_rent",
        ax=ax,
        color="#f7ec8b",
        order=median_order
    )

    # Region median
    region_median = subset["cold_rent"].median()


    # Overall median (red dashed line)
    ax.axhline(
        overall_median,
        linestyle='--',
        color='red',
        label="Overall Median"
    )

    # Region mean (blue dashed line)
    ax.axhline(
        region_median,
        linestyle='--',
        color='blue',
        label="Region Median"
```

```
    )

    ax.set_title(region)
    ax.tick_params(axis='x', rotation=45)

    ax.legend()

plt.tight_layout()
plt.show()
```


West
Overall Median
Region Median
cold_rent
0
1000
2000
3000
4000
5000
6000
Grunewald
Steglitz
Spandau
Staaken
Tiergarten
Friedenau
Dahlem
Zehlendorf
Charlottenburg
Wilmersdorf
Wannsee
Lichterfelde
Siemensstadt
Nikolassee
district


East
Overall Median
Region Median
cold_rent
0
1000
2000
3000
4000
5000
Oberschöneweide
Alt-Treptow
Marzahn
Blankenburg
Niederschöneweide
Lichtenberg
Friedrichshagen
Hellersdorf
Hohenschönhausen
Buch
Friedrichshain
Köpenick
Biesdorf
Mahlsdorf
Grünau
district


North
Overall Median
Region Median
cold_rent
0
1000
2000
3000
4000
5000
Wittenau
Reinickendorf
Wedding
Pankow
Gesundbrunnen
Tegel
Prenzlauer Berg
Niederschönhausen
Hermsdorf
Weißensee
Französisch Buchholz
district

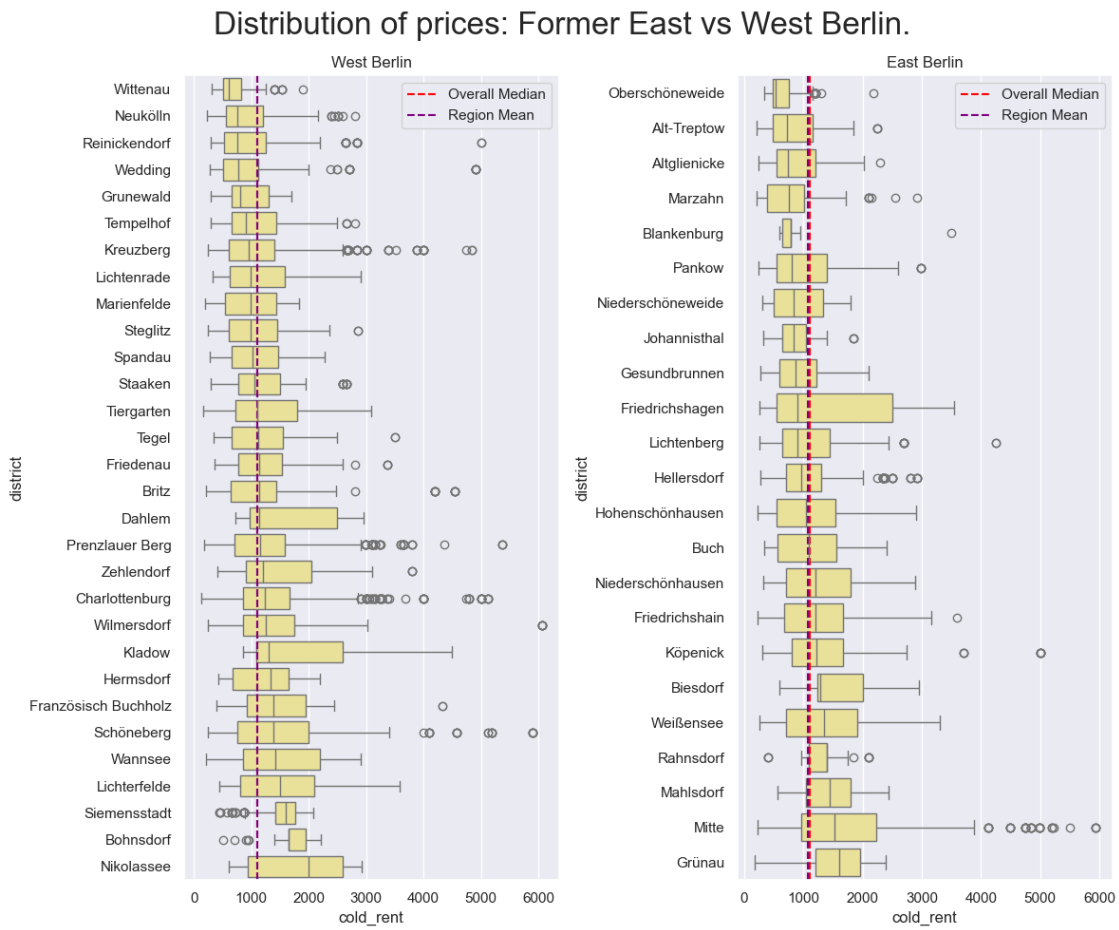

South/Central
Overall Median
Region Median
cold_rent
0
1000
2000
3000
4000
5000
6000
Altglienicke
Neukölln
Johannisthal
Tempelhof
Kreuzberg
Lichtenrade
Marienfelde
Britz
Kladow
Schöneberg
Rahnsdorf
Mitte
Bohnsdorf
district

By splitting Berlin into four regions: north, south, west and east we observe more easily the distribution of each district in each region. West region: Most of district are assosiated with higher rent overall in Berlin with only Grunewald, Steglitz and Spandau being below the overall avarage, while Nikolassee leads the western part followed by Siemensstadt, Lichterfelde and Wannsee. This region has a much more higher median compared to overall medain. East region: In this region most of the districts tend to have lower distribution of prices compared to the avarage, with only 5 regions having a higher median than the global median. District of Grünau and Mahlsdorf have a very high distribution of prices, while Friedrichshagen offers a much more wider distribution compared to rest. This region, on contradiction of the west region, has a much lower median compared to overall median. North region: It has 3 districts with a higher median than the avarage, with Französisch Buchholz being associated with the highest prices and Wittenau being associated with the lowest. Just like the east, this region has a lower distribution of prices compared to overall distribution. South region: Compared to the other 3 regions, the south has a more stable distribution of prices with 5 districts having a relatively higher distribution. One could argue that Mitte should be not be part of the south and by excluding that and Bohnsdorf we see almost a somewhat uniform distribution of prices in this region. This region has a very similar median than the overall median.

[59]:
```
west = [
    "Charlottenburg", "Wilmersdorf", "Spandau", "Zehlendorf", "Dahlem",
    "Grunewald", "Wannsee", "Steglitz", "Lichterfelde", "Friedenau",␣
 ↪"Siemensstadt",
    "Hermsdorf", "Tegel", "Wittenau", "Reinickendorf",
    "Neukölln", "Kreuzberg", "Schöneberg", "Tempelhof", "Britz",
    "Lichtenrade", "Marienfelde", "Kladow", "Tiergarten", "Wedding",
    "Prenzlauer Berg", "Staaken", "Nikolassee", "Französisch Buchholz",␣
 ↪"Bohnsdorf"
]

east = [
    "Friedrichshain", "Lichtenberg", "Hohenschönhausen", "Marzahn",␣
 ↪"Hellersdorf",
    "Köpenick", "Friedrichshagen", "Oberschöneweide", "Niederschöneweide",
    "Alt-Treptow", "Grünau", "Mahlsdorf", "Biesdorf", "Blankenburg", "Buch",
    "Pankow", "Weißensee", "Niederschönhausen", "Gesundbrunnen",
    "Mitte", "Johannisthal", "Altglienicke", "Rahnsdorf"
]

region_map = {
    "West": west,
    "East": east,
}
```

[60]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 10))

overall_median = df["cold_rent"].median()
overall_mean = df["cold_rent"].mean()

for i, (ax, region) in enumerate(zip(axes, region_map.keys())):

    subset = df[df["district"].isin(region_map[region])]

    median_order = (
        subset.groupby("district")["cold_rent"]
        .median()
        .sort_values()
        .index
    )

    # Horizontal boxplot
    sns.boxplot(
        data=subset,
        x="cold_rent",
        y="district",
        order=median_order,
        ax=ax,
        color="#f7ec8b"
    )

    region_median = subset["cold_rent"].median()


    # Overall median (vertical line)
    ax.axvline(
        overall_median,
        linestyle="--",
        color="red",
        label="Overall Median",
    )

    # Region Median
    ax.axvline(
        region_median,
        linestyle="--",
        color="purple",
        label="Region Mean",
    )

    ax.set_title(f"{region} Berlin")
```

```
    ax.legend()

fig.suptitle("Distribution of prices: Former East vs West Berlin.",size=24)
plt.subplots_adjust(hspace=0.45, top=0.92)
plt.tight_layout()
plt.show()
```



[61]:
```
west_portion = len(df[df["district"].isin(region_map["West"])]) / len(df)
print(f"West portion: {west_portion*100:.3f}%")
print(f"East portion: {(1-west_portion)*100:.3f}%")
print("-"*50)
print(f"Overall median: {df["cold_rent"].median()}")
print(f"West median: {df[df["district"].isin(region_map["West"])]["cold_rent"].
 ↪median()}")
print(f"East median: {df[df["district"].isin(region_map["East"])]["cold_rent"].
 ↪median()}")
print("-"*50)
```

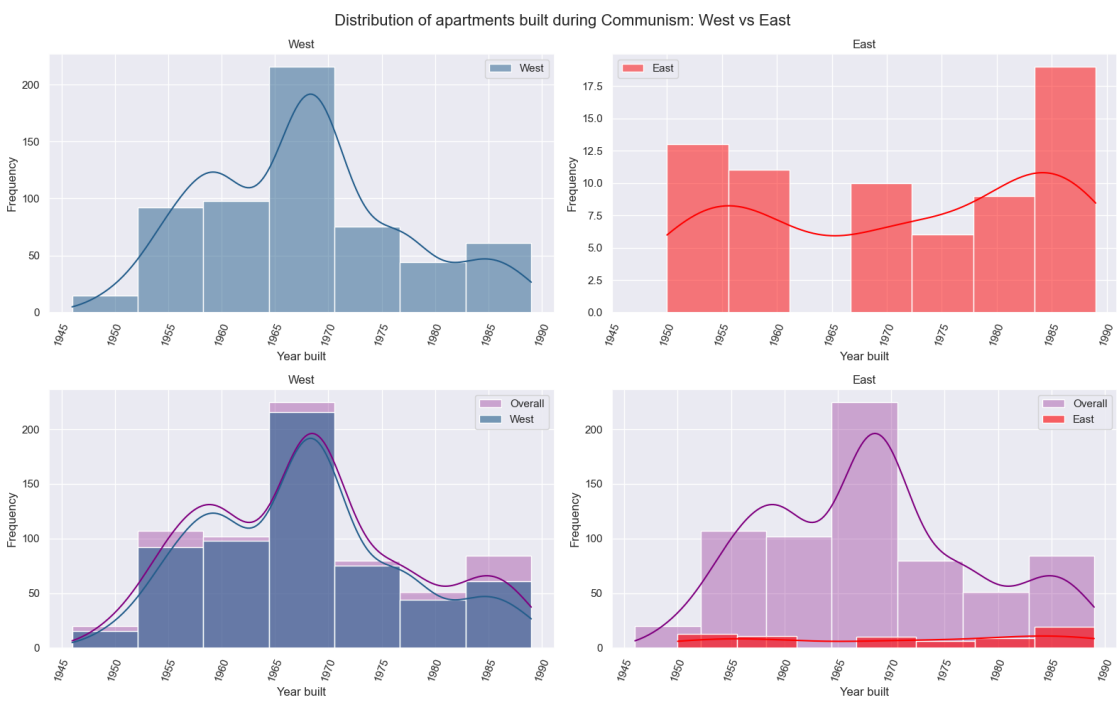
```
print(f"Overall mean: {df["cold_rent"].mean():.3f}")
print(f"West mean: {df[df["district"].isin(region_map["West"])]["cold_rent"].
 ↪mean():.3f}")
print(f"East mean: {df[df["district"].isin(region_map["East"])]["cold_rent"].
 ↪mean():.3f}")
```

```
West portion: 58.567%
East portion: 41.433%
--------------------------------------------------
Overall median: 1100.0
West median: 1100.0
East median: 1069.0
--------------------------------------------------
Overall mean: 1235.788
West mean: 1235.968
East mean: 1235.535
```

When comparing the statistics of former East Berlin and West Berlin, there is barely any difference. When it comes to distribution the West has a slightly higher distribution and a slightly higher avarage. Districts like Grünau and Mitte, which are very expensive bring up the overall statistics of the eastern part.

[62]:
```
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
sns.set(style="darkgrid")

ticks = range(1945, 1991, 5)
overall = df[(df["year_built"] >= 1945) & (df["year_built"] < 1990)]

region_colors = {
    "West": "#245e8c",
    "East": "red",
}

for i, region in enumerate(region_map.keys()):
    subset = df[df["district"].isin(region_map[region])]
    filtered = subset[(subset["year_built"] >= 1945) & (subset["year_built"] <␣
 ↪1990)]
    color = region_colors.get(region, "steelblue")  # fallback color

    # Row 1: region only
    sns.histplot(filtered["year_built"], bins=7, kde=True, ax=axes[0, i],
                 color=color, label=region)
    axes[0, i].set_xticks(ticks)
    axes[0, i].set_title(region)
    axes[0, i].set_xlabel('Year built')
    axes[0, i].set_ylabel('Frequency')
    axes[0, i].tick_params(axis='x', rotation=70)
    axes[0, i].legend()
```

```
    # Row 2: region + overall
    sns.histplot(overall["year_built"], bins=7, kde=True, ax=axes[1, i],
                 color="purple", alpha=0.3, label="Overall")
    sns.histplot(filtered["year_built"], bins=7, kde=True, ax=axes[1, i],
                 color=color, alpha=0.6, label=region)
    axes[1, i].set_xticks(ticks)
    axes[1, i].set_title(region)
    axes[1, i].set_xlabel('Year built')
    axes[1, i].set_ylabel('Frequency')
    axes[1, i].tick_params(axis='x', rotation=70)
    axes[1, i].legend()

plt.suptitle("Distribution of apartments built during Communism: West vs East",␣
 ↪size=16)
plt.tight_layout()
plt.show()
```



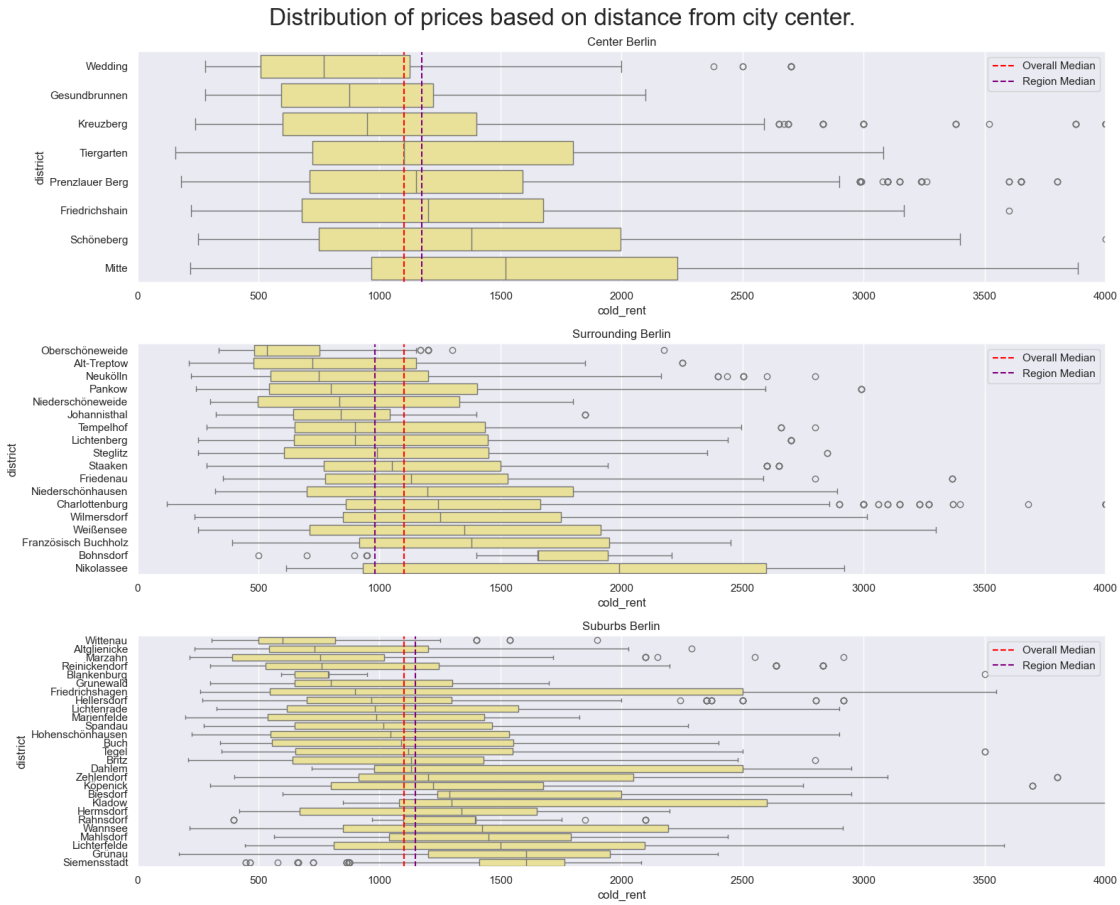
This time we can notice a huge difference, with the West dominating the East. The effect of communism is clear, with the West having the majory of apartments built during communism and the East barely having any constructions.

[63]:
```
center = [
    "Mitte",
    "Kreuzberg",
```

```
    "Friedrichshain",
    "Schöneberg",
    "Gesundbrunnen",
    "Tiergarten",
    "Prenzlauer Berg",
    "Wedding",
]

surrounding = [
    "Charlottenburg", "Wilmersdorf",
    "Pankow", "Weißensee", "Niederschönhausen",
    "Tempelhof", "Steglitz", "Friedenau",
    "Lichtenberg", "Alt-Treptow", "Neukölln",
    "Johannisthal", "Oberschöneweide", "Niederschöneweide", "Staaken",
    "Nikolassee","Bohnsdorf","Französisch Buchholz"
]

suburbs = [
    "Spandau", "Zehlendorf", "Dahlem", "Grunewald", "Wannsee",
    "Lichterfelde", "Siemensstadt",
    "Hermsdorf", "Tegel", "Wittenau", "Reinickendorf",
    "Britz", "Lichtenrade", "Marienfelde", "Kladow",
    "Hohenschönhausen", "Marzahn", "Hellersdorf",
    "Köpenick", "Friedrichshagen", "Grünau",
    "Mahlsdorf", "Biesdorf", "Blankenburg", "Buch",
    "Altglienicke", "Rahnsdorf"
]

region_map = {
    "Center": center,
    "Surrounding": surrounding,
    "Suburbs": suburbs
}
```

[64]:
```
fig, axes = plt.subplots(3, 1, figsize=(16, 13))

overall_median = df["cold_rent"].median()
overall_mean = df["cold_rent"].mean()

for i, (ax, region) in enumerate(zip(axes, region_map.keys())):

    subset = df[df["district"].isin(region_map[region])]

    median_order = (
        subset.groupby("district")["cold_rent"]
        .median()
        .sort_values()
```

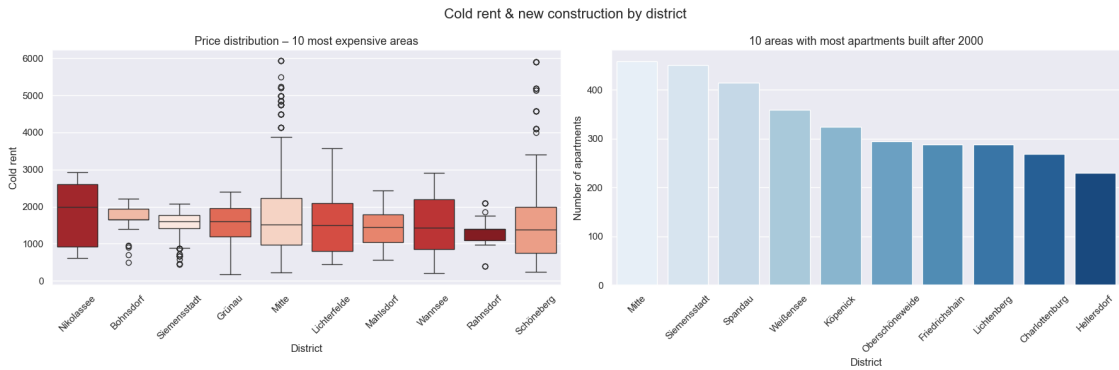
```
        .index
    )

    sns.boxplot(
        data=subset,
        x="cold_rent",
        y="district",
        order=median_order,
        ax=ax,
        color="#f7ec8b"
    )

    region_median = subset["cold_rent"].median()

    ax.axvline(overall_median, linestyle="--", color="red", label="Overall␣
 ↪Median")
    ax.axvline(region_median, linestyle="--", color="purple", label="Region␣
 ↪Median")

    ax.set_title(f"{region} Berlin")

    ax.set_xlim(0, 4000)

    ax.legend()

fig.suptitle("Distribution of prices based on distance from city center.
 ↪",size=24)
plt.tight_layout()
plt.show()
```



Here the results are very meanigfull economically, with areas near the city center being associated with higher distribution of rent prices compared to other part. The difference between overall distribution and avarage is huge. While surrounding districts have a visibly lower price distribution than overall distribution, the same thing doesn't apply to suburban areas whose distribution is actually slightly larger. This could be due to the fact that it contains a few districts with a very high price distribution like Siemensstadt, Grünau, Lichterfelde, Mahlsdorf, Wannsee, Rahnsdorf, Hermsdorf and Kladow. How is it possible that these areas very far from the city center have such high prices? This could be due to the fact that these are very new constructions to these areas and new constructions tend to have better conditions and therefore higher rent prices. Let's test wether this is true or not.

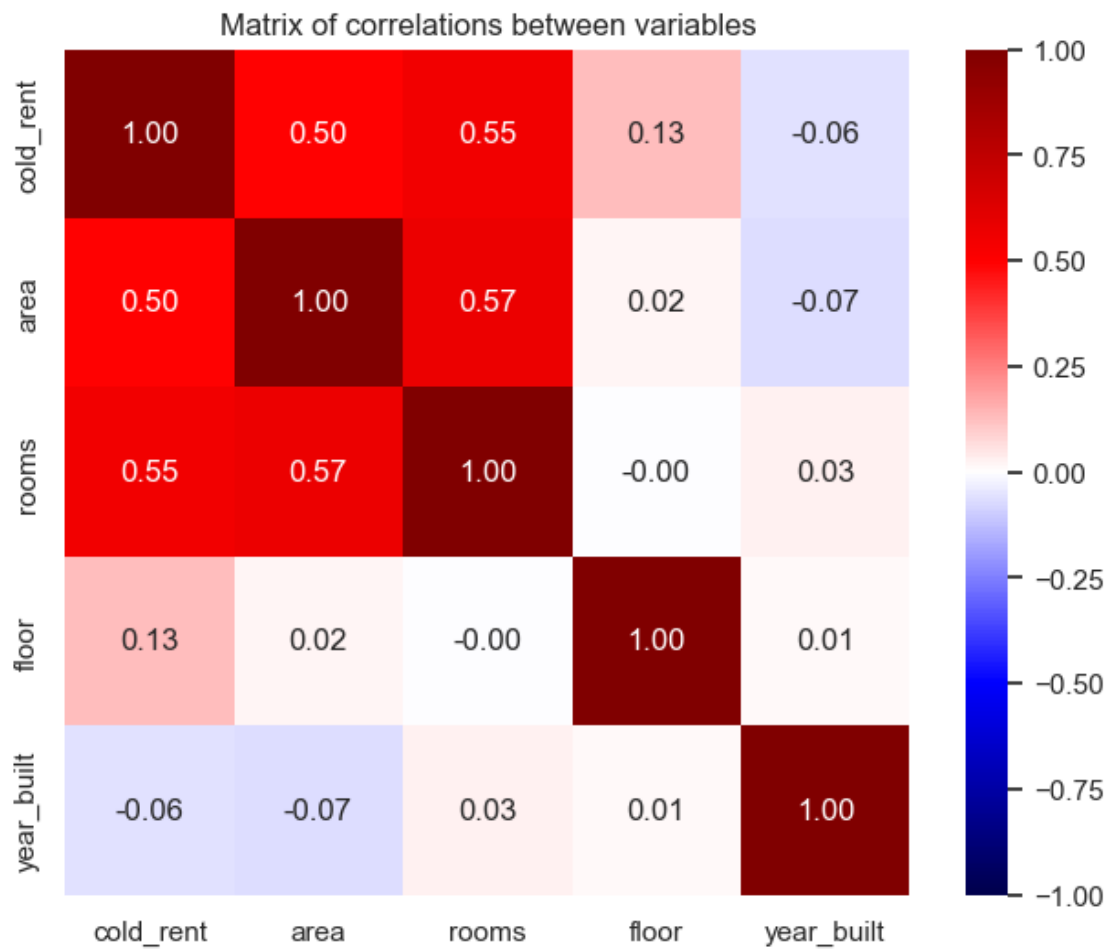
[65]:
```
fig, axes = plt.subplots(1, 2, figsize=(18, 6))
sns.set(style="darkgrid")

# 10 most expensive areas by median price
top_expensive = (
    df.groupby("district")["cold_rent"]
    .median()
    .nlargest(10)
```

```
    .index
)

# 10 areas with most apartments built after 2000
new_counts = (
    df[df["year_built"] > 2000]
    .groupby("district")
    .size()
    .nlargest(10)
    .reset_index(name="count")
)

# Plot 1: price distribution for 10 most expensive areas
sns.boxplot(
    data=df[df["district"].isin(top_expensive)],
    x="district", y="cold_rent",
    hue="district", legend=False,
    order=top_expensive,
    ax=axes[0], palette="Reds"
)
axes[0].set_title("Price distribution - 10 most expensive areas", size=13)
axes[0].set_xlabel("District")
axes[0].set_ylabel("Cold rent")
axes[0].tick_params(axis='x', rotation=45)

# Plot 2: frequency of apartments built after 2000
sns.barplot(
    data=new_counts,
    x="district", y="count",
    hue="district", legend=False,
    order=new_counts["district"],
    ax=axes[1], palette="Blues"
)
axes[1].set_title("10 areas with most apartments built after 2000", size=13)
axes[1].set_xlabel("District")
axes[1].set_ylabel("Number of apartments")
axes[1].tick_params(axis='x', rotation=45)

plt.suptitle("Cold rent & new construction by district", size=16)
plt.tight_layout()
plt.show()
```



Our theory is true for the district of Siemensstadt, while it might not apply to others since they might be small areas and therefore have a relativly smaller number of new apartments built.

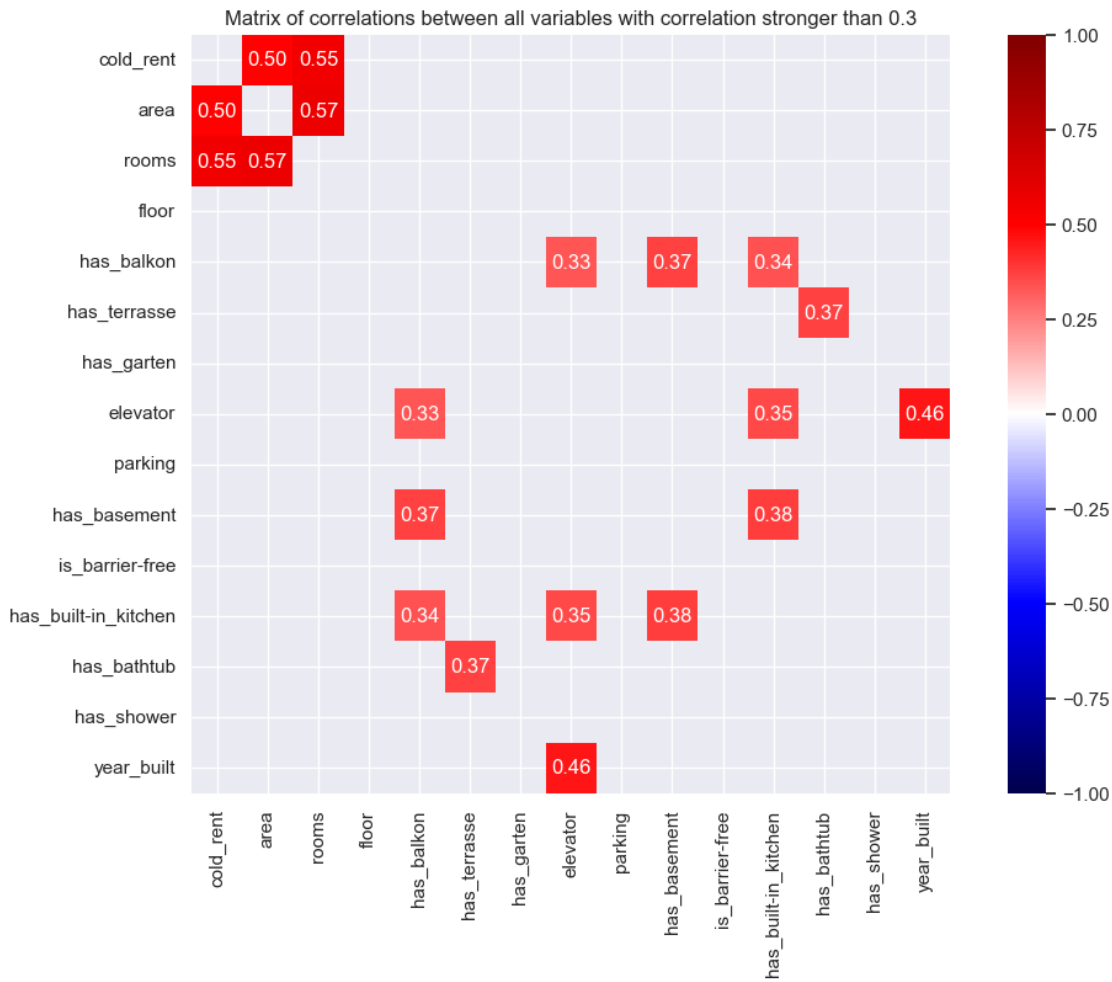
[80]:
```
cols = df.select_dtypes(include=["float64"]).columns
corr = df[cols].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", square=True, cmap="seismic", vmin=-1,␣
 ↪vmax=1)
plt.title("Matrix of correlations between variables")
plt.show()
```



There is a clear strong correlation between area and cold rent, number of rooms and cold rent and number of rooms and area, which are expected. There is a weak positive correlation between number of floors and cold rent meaning higher floors might have slightly higher prices. That makes sense if we consider the floors that have most of the occurances (0-5) and argue that higher floors like 2-5 have higher prices than groundfloor and first floor, which is meaningful economically. There is also a very weak negative correlation between year built and rent meaning that old aparments tend to have slightly lower rent prices. There is also a weak negative correlation between area and year built, which can be associated with the fact that old apartments tend to have larger areas than new apartments. Matrix of correlation provided informations that we could not grasp from some of the graphs.

[93]:
```
corr = df.corr(numeric_only=True)

# mask where absolute correlation is below 0.3
mask = (np.abs(corr) < 0.33) | (np.abs(corr) == 1.00)

plt.figure(figsize=(12, 8))
```

```
sns.heatmap(corr, annot=True, fmt=".2f", square=True,
            cmap="seismic", vmin=-1, vmax=1, mask=mask)
plt.title("Matrix of correlations between all variables with correlation␣
 ↪stronger than 0.3")
plt.show()
```



Since we have many features, let's keep track of the features with moderate to strong correlations. All of the correlations are positive. For the cold rent other than the features that we mentioned. When it comes to independent features being correlated to each other we have the following:

Having a balcony is positively correlated with having an elevator, a basement and a built-in kitchen.

Having a terrace is positively correlated with having a bathtub.

Having an elevator is positively correlated with having a balcony, a built-in kitchen and the year when the apartment was built.

Having a basement is positively correlated with having a balcony and built-in kitchen.

From these, the strongest correlation is the effect of year when the aparment was built, with having an elevator. This is very meaningful, due to the fact that old buildings don't have elevators.

[98]:
```
df.to_csv("rent_listings_final.csv", encoding="utf-8-sig", index=False)
```